

# Option Pricing under the Heston Model using Carr–Madan FFT and Neural Network Acceleration

by  
Manan Hadayat

**Bachelor Thesis**

BACHELOR OF SCIENCE IN BUSINESS ADMINISTRATION AND  
MATHEMATICAL BUSINESS ECONOMICS

at the  
COPENHAGEN BUSINESS SCHOOL

22 May 2026

Authored by: Manan Hadayat

Supervised by: Jacob Lundbeck Serup  
Part-time Lecturer, Thesis Supervisor

## ABSTRACT

This thesis investigates whether a neural network can serve as a surrogate model for the Heston option pricing function when trained on prices generated by the Carr–Madan FFT method. The Heston model captures the volatility smile observed in real markets, but pricing options under the model requires numerical methods such as the Carr–Madan FFT, which must be reevaluated for each new set of parameters and computes prices on a fixed strike grid.

We develop a feedforward neural network with 4 hidden layers of 128 neurons each, which is trained on approximately 2.5 million option prices generated by the FFT across a range of Heston parameters. We also train a second neural network directly on implied volatilities to produce smoother volatility smiles.

The results show that the network achieves small absolute pricing errors across all moneyness regions, with the lowest errors for ITM options. On the full held-out test set, the mean absolute error is 0.0041 for ITM options, 0.0061 for near ATM options, and 0.0052 for OTM options. The OTM region shows higher relative errors, which is driven by the low absolute prices rather than large absolute errors. Heat map analysis shows that the network performs well within the sampled parameter range, with the highest errors occurring for long maturities combined with high vol-of-vol. For a single option price, the neural network does not outperform Carr–Madan direct integration. However, when computing a volatility surface across 10 maturities and 20 strikes, the neural network is approximately three times as fast as the FFT, and up to 42x faster when all inputs are evaluated in a single batch.

## RESUME

Dette speciale undersøger, om et neuralt netværk kan fungere som en surrogatmodel for Hestons optionsprisfunktion, når det er trænet på priser genereret af Carr–Madan FFT metoden. Heston-modellen fanger det volatilitetssmil, der observeres i virkelige markeder, men prissætning af optioner under modellen kræver numeriske metoder som Carr–Madan FFT, der skal evalueres for hvert nyt sæt af parametre og beregner priser på et fast strike-grid.

Vi udvikler et feedforward neuralt netværk med 4 skjulte lag med 128 neuroner i hvert lag, som er trænet på ca. 2,5 millioner optionspriser genereret af FFT'en på tværs af en række Heston-parametre. Vi træner også et andet neuralt netværk direkte på implicerede volatiliteter for at producere glattere volatilitetssmil.

Resultaterne viser, at netværket opnår små absolutte prissætningsfejl på tværs af alle moneyness-regioner, med de laveste fejl for ITM-optioner. På det fulde testdatasæt er den gennemsnitlige absolutte fejl 0,0041 for ITM-optioner, 0,0061 for near ATM-optioner og 0,0052 for OTM-optioner. OTM-regionen viser højere relative fejl, hvilket skyldes de lave absolutte priser snarere end store absolutte fejl. Heat map analyse viser, at netværket klarer sig godt inden for det samlede parameterrum, hvor de højeste fejl forekommer for lange løbetider kombineret med høj vol-of-vol. For en enkelt optionspris er det neurale netværk ikke hurtigere end Carr–Madan "direct" integration. Når der derimod beregnes en hel volatilitetsflade på tværs af 10 løbetider og 20 strikes, er det neurale netværk ca. tre gange så hurtigt som FFT'en, og op til 42 gange hurtigere, når alle inputs evalueres i en enkelt batch.

# Contents

|          |                                                        |           |
|----------|--------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                    | <b>5</b>  |
| 1.1      | Motivation . . . . .                                   | 5         |
| 1.2      | Research Question . . . . .                            | 6         |
| 1.3      | Preliminaries . . . . .                                | 6         |
| 1.3.1    | European Options . . . . .                             | 6         |
| 1.3.2    | Implied Volatility and the Volatility Smile . . . . .  | 7         |
| <b>2</b> | <b>The Heston Model</b>                                | <b>9</b>  |
| 2.1      | Model Dynamics . . . . .                               | 9         |
| 2.2      | Risk-Neutral Dynamics . . . . .                        | 10        |
| 2.3      | Model Parameters and Intuition . . . . .               | 11        |
| 2.4      | Properties of the Model . . . . .                      | 12        |
| <b>3</b> | <b>The Characteristic Function</b>                     | <b>13</b> |
| 3.1      | Definition and Properties . . . . .                    | 13        |
| 3.2      | Characteristic Function of the Heston Model . . . . .  | 13        |
| 3.3      | Derivation . . . . .                                   | 14        |
| <b>4</b> | <b>The Carr–Madan FFT Method</b>                       | <b>16</b> |
| 4.1      | Fourier Transform Approach to Option Pricing . . . . . | 16        |
| 4.2      | The Carr–Madan Framework . . . . .                     | 16        |
| 4.3      | Fast Fourier Transform Algorithm . . . . .             | 18        |
| 4.4      | Damping Factor and Numerical Considerations . . . . .  | 19        |
| <b>5</b> | <b>Implementation</b>                                  | <b>20</b> |
| 5.1      | Implementation of the Carr–Madan Method . . . . .      | 20        |
| 5.2      | Validation and Numerical Stability . . . . .           | 21        |
| <b>6</b> | <b>Neural Network Acceleration</b>                     | <b>23</b> |
| 6.1      | Motivation for Neural Network Surrogates . . . . .     | 23        |
| 6.2      | Network Architecture . . . . .                         | 24        |
| 6.2.1    | Architecture Selection . . . . .                       | 25        |
| 6.3      | Training Data . . . . .                                | 26        |
| 6.4      | Training Procedure . . . . .                           | 27        |
| 6.5      | Loss Function and Optimisation . . . . .               | 29        |

|          |                                                   |           |
|----------|---------------------------------------------------|-----------|
| <b>7</b> | <b>Results</b>                                    | <b>31</b> |
| 7.1      | Carr–Madan Pricing Results . . . . .              | 31        |
| 7.2      | Neural Network Performance . . . . .              | 32        |
| 7.2.1    | Pricing Accuracy . . . . .                        | 32        |
| 7.2.2    | Generalization Across Parameter Regions . . . . . | 35        |
| 7.2.3    | Implied Volatility Analysis . . . . .             | 36        |
| 7.3      | Comparison and Benchmarking . . . . .             | 37        |
| <b>8</b> | <b>Conclusion</b>                                 | <b>40</b> |
| 8.1      | Summary of Findings . . . . .                     | 40        |
| 8.2      | Limitations . . . . .                             | 41        |
| 8.3      | Future Work . . . . .                             | 41        |
|          | <i>References</i>                                 | 42        |
| <b>A</b> | <b>Mathematical Results</b>                       | <b>44</b> |
| A.1      | Theorems . . . . .                                | 44        |
| A.2      | Derivations . . . . .                             | 45        |
| <b>B</b> | <b>Computational Results</b>                      | <b>46</b> |
| B.1      | Tables . . . . .                                  | 46        |
| B.2      | Figures . . . . .                                 | 57        |
| <b>C</b> | <b>Source Code</b>                                | <b>60</b> |
| C.1      | Carr–Madan FFT Implementation . . . . .           | 60        |
| C.2      | Neural Network Architecture . . . . .             | 62        |
| C.3      | Training Data Generation . . . . .                | 63        |

# Chapter 1

## Introduction

Option pricing plays a central role in the financial markets, with applications in risk management, hedging, and trading. Accurate option pricing is essential, as it directly affects decision making across financial markets.

One model used for option pricing is known as the Black-Scholes model, which was introduced in 1973 [1]. It is widely used as a foundation for more advanced models. Its primary limitation is that it assumes constant volatility, thus it does not align with the real market, which features stochastic volatility.

This limitation has been addressed by stochastic volatility models, which account for the stochastic nature of volatility in option pricing. A model that is widely used in the financial industry is the Heston model introduced in 1993 [2], which allows the volatility to vary over time and allowing it to reflect market dynamics more closely than the Black-Scholes model. However, the model's ability to reflect market dynamics comes at a numerical cost. As the Heston model is more complex than Black-Scholes, it requires more advanced numerical methods, which can be challenging. This leads to a need for more efficient numerical methods.

### 1.1 Motivation

A significant step toward an efficient numerical method for the Heston model was made by Carr and Madan [3], who showed that the characteristic function of the model can be combined with the Fast Fourier Transform to price European options efficiently. Previously, methods such as Monte Carlo simulation were being used, which required a high amount of computation. The error of Monte Carlo simulation decreases proportionally to  $\frac{1}{\sqrt{N}}$ , where  $N$  is the number of samples. This means that achieving 5x better accuracy it requires 25x more simulations, which results in a slow convergence speed. The Carr–Madan FFT approach avoids this by computing option prices across a grid of strike prices through the characteristic function, offering a faster alternative to Monte Carlo.

However, the Carr–Madan FFT has two key limitations. Firstly, it requires a new computation for each set of input parameters. This means that even if a single FFT evaluation is computationally efficient, the method can become computationally expensive because it must be executed for every new set of parameters, as it cannot reuse previously stored computations. Secondly, the FFT computes option prices on a fixed grid of strike prices, which means that any strike price falling between grid points will not be

calculated directly. Instead, such prices will be estimated through interpolation between two neighboring computed values, which introduces approximation error as the true pricing function is nonlinear.

A possible solution to both limitations is neural networks, which have seen significant advancements driven by increased computational power and data availability. Neural networks can be used for this purpose, as they can approximate any continuous function to arbitrary precision. In the context of option pricing, they can be used as a surrogate model that maps Heston parameters and strike prices to option prices by training on prices generated using the FFT. Once trained, the neural network is used to approximate the Heston pricing function, which gives it the ability to compute option prices in near real time. Neural networks also address the grid constraint of the FFT, as the neural network takes the strike price as a continuous input rather than being constrained to a fixed grid. This does not imply that the neural network is more accurate than FFT interpolation, but it provides a direct approximation at any desired strike without requiring a separate interpolation step.

## 1.2 Research Question

The main research question for this thesis is: Can a neural network accurately serve as a surrogate model for the Heston option pricing function when trained on prices generated by the Carr–Madan FFT method, while offering computational advantages over direct FFT evaluation?

The thesis will investigate this through the following sub-questions:

- How accurately can a neural network learn the Heston pricing function when trained on Carr–Madan FFT generated prices?
- How do the neural network and the Carr–Madan FFT method compare in terms of computational efficiency?
- Can the neural network generalize within the parameter space of the Heston model?

## 1.3 Preliminaries

In this section, we introduce two concepts that are necessary for the understanding of the coming chapters. In particular, we define European options and their payoff structure, as well as implied volatility and the volatility smile, which motivate the use of the Heston model.

### 1.3.1 European Options

A European option is a form of contract, which gives the option holder the right to either buy or sell the underlying asset  $S$  at a strike price  $K$ , but not the obligation to do so [4]. The right to buy at a strike price  $K$  is called a call option and correspondingly the right to sell at strike  $K$  is a put option. The European option can only be exercised at its maturity date (expiry date)  $T$  as opposed to American options which can be exercised at any time prior to maturity  $T$ .

As stated, the option holder is not obligated to exercise the option and therefore a rational option holder will only exercise a call option when  $S_T > K$ , and a put option is only exercised when  $S_T < K$ . If these conditions are not fulfilled, the option will expire worthless.

**Definition 1.1.** The payoff of a European option is the value realised by the option holder at maturity  $T$ .

For a call option with strike  $K$  the payoff is

$$\max(S_T - K, 0)$$

For a put option with strike  $K$  the payoff is

$$\max(K - S_T, 0)$$

This payoff structure is illustrated visually in Figure 1.1, which shows the payoff of a European call and put option at maturity  $T$ .

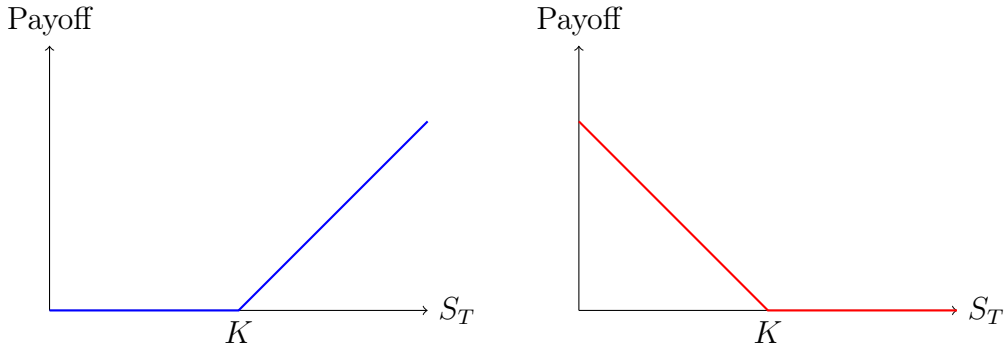


Figure 1.1: Payoff diagrams for a European call option (left) and put option (right) with strike price  $K$

### 1.3.2 Implied Volatility and the Volatility Smile

Implied volatility of an option is a measure of the volatility of an underlying asset. This measure has a direct effect on the option price, as higher implied volatility leads to options with higher premiums because a higher volatility widens the distribution of possible future asset prices, which increases the likelihood of the option expiring in the money because the sample space increases, and therefore increases its value.

The formal definition is as follows:

**Definition 1.2.** Implied volatility  $\sigma_{\text{imp}}$  is the volatility input to the Black-Scholes formula that recovers the observed market price of an option [4].

Under the Black-Scholes model, volatility is assumed to be constant, which implies that  $\sigma_{\text{imp}}$  should be the same across all strike prices  $K$  and maturities  $T$ , which produces a flat volatility surface. However, in practice this is not observed. Plotting implied volatility against the strike price  $K$  for a fixed maturity  $T$  shows a curved shape known as the

volatility smile. This curve is often skewed, which shows higher implied volatility for lower strikes, which is referred to as the volatility skew.

The existence of a volatility skew tells us that the constant volatility assumption of the Black-Scholes model is inconsistent with observed market prices. This motivates the use of stochastic volatility in Chapter 2, which will allow the volatility to vary over time and therefore being able to produce a volatility smile consistent with the real market. This concept is illustrated in Figure 1.2.

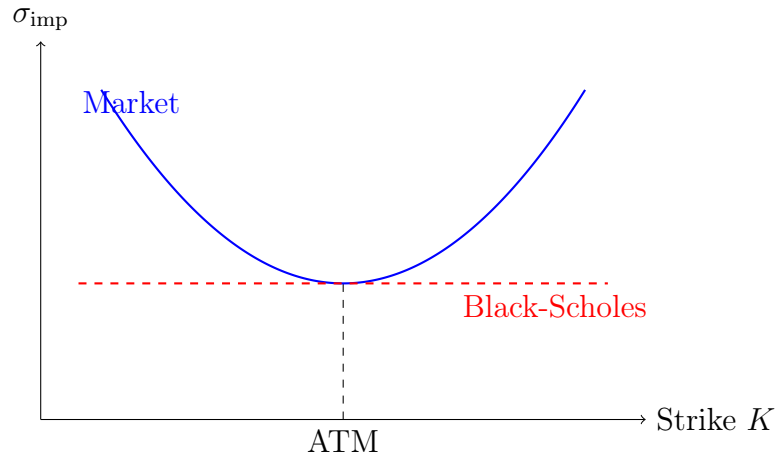


Figure 1.2: Illustration of the volatility smile showing implied volatility  $\sigma_{\text{imp}}$  against strike price  $K$  for a fixed maturity  $T$



# Chapter 2

## The Heston Model

This chapter develops the Heston model, which serves as the pricing framework throughout the thesis. We first present the model dynamics under the physical measure, followed by the transformation to risk-neutral dynamics under which option prices are computed. The chapter then discusses the model parameters and their financial interpretation, along with key properties of the model that are relevant to the pricing methods developed in the following chapters.

### 2.1 Model Dynamics

The Heston model, introduced by Heston [2], describes the evolution of the price of an asset  $S$  and the variance  $v$  through a system of stochastic differential equations (SDEs)

$$dS_t = \mu_t S_t dt + \sqrt{v_t} S_t dZ_1 \quad (2.1)$$

$$dv_t = \kappa(\theta - v_t) dt + \sigma \sqrt{v_t} dZ_2 \quad (2.2)$$

where  $Z_1$  and  $Z_2$  are the standard Wiener processes with instantaneous correlation given as  $dZ_1 dZ_2 = \rho dt$ .

The first equation describes the asset price dynamics, where the first term is called the drift, which represents the predictable part of the movement of the asset.  $\mu_t$  is the instantaneous drift rate of the asset price, representing the expected proportional growth of the asset per unit of time. The second term is the random shock, which represents the unpredictable change in the asset price. The size of this random shock depends on the variance  $v_t$ , which is stochastic, and  $dZ_1$  introduces random fluctuations in the asset price.

The second equation describes the evolution of the variance  $v_t$ , where the first term is called the drift, which represents the predictable part of the movement in the variance. The variance process follows the Cox-Ingersoll-Ross (CIR) process [5], which features mean reversion. The drift term  $\kappa(\theta - v_t)$  is mean reverting, which means that when the variance is above its long run mean  $\theta$ , the drift becomes negative and pulls the variance back down, and when  $v_t$  is below  $\theta$ , the drift becomes positive and pushes the variance back up. Also note that  $\kappa$  controls the speed of this reversion. The second term is the random shock, which represents the unpredictable change in the variance. The size of this random shock depends on the current variance level  $v_t$ , and  $dZ_2$  introduces random fluctuations in the variance.

For later use, it is convenient to consider the log price process  $\log S_t$ , as it simplifies the later calculations. Applying Itô's lemma on equation (2.1) with  $x_t = \log S_t$  we get that

$$d \log S_t = \left( \mu_t - \frac{1}{2} v_t \right) dt + \sqrt{v_t} dZ_1 \quad (2.3)$$

The model has an initial condition, which states that  $S_0 > 0$  and  $v_0 > 0$ . This ensures that the model is well-defined, as asset prices cannot be negative and variances are not defined as negative. The positivity of  $S_0$  is preserved by the structure of the asset price dynamics, but the same does not hold for  $v_t$ . Therefore, it requires an additional condition on the model parameters, which will be discussed in Section 2.4.

## 2.2 Risk-Neutral Dynamics

In the previous section, we introduced the Heston model dynamics under the real-world measure  $\mathbb{P}$  with  $\mu_t$ , which presents two challenges for pricing, such as not being observable in the market and investors have different expectations for  $\mu_t$ . We therefore shift to the risk-neutral measure  $\mathbb{Q}$ , where  $\mu_t$  is replaced with the risk-neutral drift  $r - q$ , where  $r$  is the risk-free rate and  $q$  is the continuous dividend yield. This measure change can be done by using Girsanov's Theorem A.1 where we set  $d = 2$  as the Heston model has two Brownian motions  $Z_1$  and  $Z_2$ .

We define the market price of risk as

$$\Theta_1(t) = \frac{\mu_t - (r - q)}{\sqrt{v_t}} \quad (2.4)$$

$$\Theta_2(t) = \frac{\lambda \sqrt{v_t}}{\sigma} \quad (2.5)$$

where  $\Theta_1(t)$  is the market price of equity risk, which is uniquely determined by no-arbitrage.  $\Theta_2(t)$  is market price of variance risk, which is not uniquely determined as the variance is not a traded asset, it has no no-arbitrage condition.  $\Theta_2(t)$  is determined by following Heston [2], where it is assumed that the volatility risk premium takes the form  $\lambda(S_t, v_t, t) = \lambda v_t$ , where  $\lambda$  is a constant. The full derivation of  $\Theta_1(t)$  and  $\Theta_2(t)$  is given in Appendix A.2.1.

We can now define new Brownian motions under  $\mathbb{Q}$  as

$$\tilde{Z}_1(t) = Z_1(t) + \int_0^t \Theta_1(u) du \quad (2.6)$$

$$\tilde{Z}_2(t) = Z_2(t) + \int_0^t \Theta_2(u) du \quad (2.7)$$

We now substitute these into the Heston model dynamics, which gives us the risk-neutral processes

$$dS_t = (r - q)S_t dt + \sqrt{v_t}S_t d\tilde{Z}_1 \quad (2.8)$$

$$dv_t = \kappa^*(\theta^* - v_t) dt + \sigma \sqrt{v_t} d\tilde{Z}_2 \quad (2.9)$$

where  $\kappa^* = \kappa + \lambda$ ,  $\theta^* = \frac{\kappa\theta}{\kappa + \lambda}$ , and  $\tilde{Z}_1$  and  $\tilde{Z}_2$  remain correlated with  $d\tilde{Z}_1 d\tilde{Z}_2 = \rho dt$ .

Here  $q$  denotes the continuous dividend yield. If  $q = 0$ , the risk-neutral drift reduces to  $r$ .

By the risk-neutral pricing formula [6, Equation 5.2.31], the price of a derivative security  $V$  with payoff  $V_T$  at maturity  $T$  is given by

$$V(t) = \mathbb{E}^{\mathbb{Q}}[e^{-\int_t^T R(u) du} V_T \mid \mathcal{F}_t]$$

We use that the variable interest rate is constant, meaning  $R(u) = r$ , which gives us  $\int_t^T R(u) du = \int_t^T r du = r(T - t)$ . We therefore have that the price of a derivative security is given by

$$V(t) = e^{-r(T-t)} \mathbb{E}^{\mathbb{Q}}[V_T \mid \mathcal{F}_t] \quad (2.10)$$

## 2.3 Model Parameters and Intuition

We begin by presenting the model parameters in a table overview, which can be seen below.

| Parameter | Description               | Interpretation                                      |
|-----------|---------------------------|-----------------------------------------------------|
| $\kappa$  | Speed of mean reversion   | Controls how fast $v_t$ reverts to $\theta$         |
| $\theta$  | Long-run mean of variance | The level $v_t$ reverts to in the long run          |
| $\sigma$  | Volatility of volatility  | Controls the variability of $v_t$ itself            |
| $\rho$    | Correlation               | Correlation between asset price and variance shocks |
| $v_0$     | Initial variance          | The starting value of the variance process          |
| $\lambda$ | Variance risk premium     | Investor compensation for taking volatility risk    |

Table 2.1: Heston model parameters and their interpretation

Understanding the relationship between the parameters and the Heston model is important, because they are used as input parameters in the neural network defined in later chapters.

The parameters  $\kappa$  and  $\theta$  control the mean reversion of the variance process, as discussed in Section 2.1. A larger value of  $\kappa$  implies that the variance reverts faster towards  $\theta$ , and correspondingly a smaller value of  $\kappa$  implies slower reversion, which allows the variance to differ from the long-run mean of variance for a longer period. A larger value of  $\theta$  implies a higher long-run variance, which leads to higher expected volatility, and therefore option prices increase across all maturities.

The volatility of volatility  $\sigma$  controls how much  $v_t$  can vary. A large value of  $\sigma$  implies that the variance is highly fluctuating, which increases the kurtosis of the return distribution and raises the prices of options far out of the money. Correspondingly, a small value of  $\sigma$  implies that the variance is more stable, leading to return distributions closer to the normal distribution.

$\rho$  is the correlation between  $Z_1$  and  $Z_2$ , which means that it tells us the relationship between shocks to the asset price and shocks to the variance. In practice,  $\rho$  is typically negative, which captures the leverage effect, meaning that when the asset price falls, the variance tends to increase. A negative  $\rho$  produces a negatively skewed return distribution, while a positive  $\rho$  produces a positively skewed return distribution. This skewness affects option prices, as shown by Heston [2, Figure 2], where a negative  $\rho$  raises the prices of

in the money (ITM) call options, and a positive  $\rho$  raises the prices of out of the money (OTM) call options. Note that in this thesis, we only consider  $\rho \leq 0$ , since we only focus on equity options.

$v_0$  is the initial variance of the asset at time  $t = 0$ . This shows how the variance process behaves in the short term, where a larger  $v_0$  implies higher volatility, which raises the price of options with short maturity. This effect disappears for longer maturities as the variance reverts towards  $\theta$ .

$\lambda$  is the variance risk premium, which represents the compensation investors require for bearing volatility risk. As we discussed in Section 2.2, we have that  $\lambda$  affects the risk-neutral parameters through  $\kappa^* = \kappa + \lambda$  and  $\theta^* = \frac{\kappa\theta}{\kappa+\lambda}$ . In practice, we have that  $\lambda = 0$  [7], which implies that  $\kappa^* = \kappa$  and  $\theta^* = \theta$ . This assumption will be used throughout the thesis.

## 2.4 Properties of the Model

As mentioned in Section 2.1 we cannot guarantee the positivity of  $v_t$  by the dynamics and therefore we require an additional condition. This condition is known as the Feller condition.

**Definition 2.1.** The Feller condition states that

$$2\kappa\theta > \sigma^2$$

Assuming that the Feller condition is satisfied, the variance process  $v_t$  will be strictly positive for all  $t > 0$ . If the condition is violated, the variance can reach zero, which can cause numerical instability in pricing and simulation.

We now formalize the mean-reverting behavior of  $v_t$  discussed in Section 2.1. Conditioned on  $v_s$  with  $s < t$ , the expected value of  $v_t$  is given by

$$\mathbb{E}[v_t \mid v_s] = \theta + (v_s - \theta)e^{-\kappa(t-s)},$$

which shows that  $v_t$  reverts towards  $\theta$  at an exponential rate controlled by  $\kappa$ . Over a finite time horizon, the  $v_t$  follows a non-central chi-squared distribution, and as  $t \rightarrow \infty$ , the variance process converges to a stationary gamma distribution with mean  $\theta$  [7].

# Chapter 3

## The Characteristic Function

This chapter introduces the characteristic function, which is crucial for the Carr–Madan FFT method for option pricing. We begin by defining the characteristic function and presenting its key properties, which will be followed by the characteristic function of the Heston model. Eventually, we will derive the characteristic function from the risk-neutral dynamics presented in Chapter 2.

### 3.1 Definition and Properties

The characteristic function of a random variable  $X$  is defined as the expected value of  $e^{iuX}$ , where  $u \in \mathbb{R}$  and  $i$  is the imaginary unit.

**Definition 3.1.** Let  $X$  be a random variable defined on  $(\Omega, \mathcal{A}, P)$  with the probability density function  $f_X(x)$ . The characteristic function of  $X$  is  $\phi : \mathbb{R} \rightarrow \mathbb{C}$  given by

$$\phi(u) = \mathbb{E}[e^{iuX}] = \int_{-\infty}^{\infty} e^{iux} f_X(x) dx \quad (3.1)$$

The characteristic function has several key properties, one of which is existence. It always exists for all  $u \in \mathbb{R}$  because the complex exponential function  $e^{iuX}$  has a modulus of  $|e^{iuX}| = 1$ , which implies that it is bounded. Therefore, the integral of the exponential function is well defined. We also have that if we assume there are two random variables  $X$  and  $Y$  and both have the same characteristic function, then they both have the same probability distribution. This shows us that a probability distribution is uniquely determined by its characteristic function.

Another key property of the characteristic function relevant to this thesis is that it is closely related to the Fourier transform of the probability density function. This makes the characteristic function notably useful for numerical methods based on the Fast Fourier Transform, which will be discussed in Chapter 4.

### 3.2 Characteristic Function of the Heston Model

From equation (2.10) we have the price of a derivative security, which requires the probability density function of  $S_T$ . The problem for the Heston model is that we do not have the probability density function in closed form, unlike in the Black-Scholes model.

However, Heston [2] showed that the characteristic function of  $\log S_T$  can be derived in closed form.

We have from Heston that the characteristic function of  $\log S_T$  has the form

$$f_j(u; x, v) = \exp \left( C_j(\tau, u) + D_j(\tau, u)v + iux \right) \quad (3.2)$$

where  $x = \log S_t$ ,  $v = v_t$ ,  $\tau = T - t$  is the time to maturity.

We have that  $D_j$  and  $C_j$  are given by

$$C_j(\tau, u) = (r - q)iu\tau + \frac{\kappa\theta}{\sigma^2} \left[ (b_j - \rho\sigma iu + d_j)\tau - 2 \log \left( \frac{1 - g_j e^{d_j \tau}}{1 - g_j} \right) \right] \quad (3.3)$$

$$D_j(\tau, u) = \frac{b_j - \rho\sigma iu + d_j}{\sigma^2} \left( \frac{1 - e^{d_j \tau}}{1 - g_j e^{d_j \tau}} \right) \quad (3.4)$$

with

$$g_j = \frac{b_j - \rho\sigma iu + d_j}{b_j - \rho\sigma iu - d_j}, \quad d_j = \sqrt{(\rho\sigma iu - b_j)^2 - \sigma^2(2u_j iu - u^2)} \quad (3.5)$$

and the parameters  $u_j$  and  $b_j$  are defined as

$$u_1 = \frac{1}{2}, \quad u_2 = -\frac{1}{2}, \quad b_1 = \kappa - \rho\sigma, \quad b_2 = \kappa$$

These equations are defined for  $j = 1, 2$ . Note that we have used  $\lambda = 0$  as discussed in Section 2.3, which simplifies the expressions for  $b_1$  and  $b_2$  compared to the equations Heston presents. Also note the distinction that  $u$  is the Fourier variable, whereas  $u_j$  are the constants.

The Heston model has two characteristic functions denoted as  $f_1$  and  $f_2$ , which correspond to the two risk-neutral probabilities  $P_1$  and  $P_2$  used in the pricing formula, which will be discussed in detail in Chapter 4.

### 3.3 Derivation

In this section we will derive the characteristic function in equation (3.2). Note that  $f_1$  and  $f_2$  are under different probability measures, where  $f_1$  is under the measure  $\mathbb{Q}^S$  where the asset is numeraire and  $f_2$  is under the risk-neutral measure  $\mathbb{Q}$ . The derivation below is presented for  $f_2$ , and  $f_1$  follows from a similar derivation under  $\mathbb{Q}^S$ . Heston [2] and Rouah [7] provide the details on the measure change.

As previously noted, we will work with  $x_t = \log S_t$ , whose risk-neutral dynamics are found using equation (2.3) and equation (2.8):

$$dx_t = \left( r - q - \frac{1}{2}v_t \right) dt + \sqrt{v_t} d\tilde{Z}_1,$$

and note that  $dv_t$  remains unchanged as stated previously. We have the characteristic function

$$f_2(u; x, v) = \mathbb{E}^{\mathbb{Q}}[e^{iux_T} \mid x_t = x, v_t = v]$$

which is a conditional expectation under  $\mathbb{Q}$  and therefore a martingale. Applying Ito's lemma to  $f_2$ , we get that

$$df_2 = \frac{\partial f_2}{\partial t} dt + \frac{\partial f_2}{\partial x} dx_t + \frac{\partial f_2}{\partial v} dv_t + \frac{1}{2} \frac{\partial^2 f_2}{\partial x^2} v_t dt + \rho \sigma v_t \frac{\partial^2 f_2}{\partial x \partial v} dt + \frac{1}{2} \sigma^2 v_t \frac{\partial^2 f_2}{\partial v^2} dt$$

We can now substitute the risk-neutral dynamics  $dx_t$  and  $dv_t$  and set the drift to zero, as it is a martingale condition. This gives us that

$$\frac{\partial f_2}{\partial t} + \frac{1}{2} v \frac{\partial^2 f_2}{\partial x^2} + \rho \sigma v \frac{\partial^2 f_2}{\partial x \partial v} + \frac{1}{2} \sigma^2 v \frac{\partial^2 f_2}{\partial v^2} + \left( r - q - \frac{1}{2} v \right) \frac{\partial f_2}{\partial x} + \kappa(\theta - v) \frac{\partial f_2}{\partial v} = 0 \quad (3.6)$$

Changing variables from  $t$  to  $\tau = T - t$  gives us  $\frac{\partial}{\partial t} = -\frac{\partial}{\partial \tau}$ . Using the notation  $a = \kappa\theta$  and the constants  $u_2$  and  $b_2$  as defined in Section 3.2, which are obtained by matching the drift coefficients of the PDE under  $\mathbb{Q}$ , we get

$$\frac{\partial f_2}{\partial \tau} = \frac{1}{2} v \frac{\partial^2 f_2}{\partial x^2} + \rho \sigma v \frac{\partial^2 f_2}{\partial x \partial v} + \frac{1}{2} \sigma^2 v \frac{\partial^2 f_2}{\partial v^2} + (r - q + u_2 v) \frac{\partial f_2}{\partial x} + (a - b_2 v) \frac{\partial f_2}{\partial v} \quad (3.7)$$

which is subject to the terminal condition  $f_2(u; x, v; 0) = e^{iux}$  because  $\tau = 0$  implies  $t = T$ , which gives us  $\mathbb{E}^{\mathbb{Q}}[e^{iux_T} \mid x_T = x] = e^{iux}$ . We guess a solution of the exponential affine form

$$f_2(u; x, v) = \exp(C_2(\tau, u) + D_2(\tau, u)v + iux) \quad (3.8)$$

where  $\tau = T - t$ . By computing the partial derivatives of equation (3.8) and substituting into equation (3.7) gives us

$$\frac{\partial C_2}{\partial \tau} + \frac{\partial D_2}{\partial \tau} v = -\frac{1}{2} v u^2 + \rho \sigma v i u D_2 + \frac{1}{2} \sigma^2 v D_2^2 + (r - q + u_2 v) i u + (a - b_2 v) D_2$$

We can now separate the terms involving  $v$  from those that do not, which gives us two ordinary differential equations

$$\frac{\partial D_2}{\partial \tau} = \frac{1}{2} \sigma^2 D_2^2 + (\rho \sigma i u - b_2) D_2 - \frac{1}{2} u^2 + u_2 i u \quad (3.9)$$

$$\frac{\partial C_2}{\partial \tau} = (r - q) i u + a D_2 \quad (3.10)$$

which is subject to the initial conditions  $C_2(0, u) = 0$  and  $D_2(0, u) = 0$ . We have that equation (3.9) is a Riccati ODE with constant coefficients, which gives us the closed-form solution

$$D_2(\tau, u) = \frac{b_2 - \rho \sigma i u + d_2}{\sigma^2} \left( \frac{1 - e^{d_2 \tau}}{1 - g_2 e^{d_2 \tau}} \right)$$

where  $d_2$  and  $g_2$  are defined in Section 3.2. We can now substitute  $D_2$  into equation (3.10) and integrate from 0 to  $\tau$ , which gives us

$$C_2(\tau, u) = (r - q) i u \tau + \frac{a}{\sigma^2} \left[ (b_2 - \rho \sigma i u + d_2) \tau - 2 \log \left( \frac{1 - g_2 e^{d_2 \tau}}{1 - g_2} \right) \right]$$

We have now completed the derivation of the characteristic function.

# Chapter 4

## The Carr–Madan FFT Method

In this chapter, we will use the characteristic function derived in the previous chapter to implement the Fast Fourier Transform method introduced by Carr and Madan [3]. We first present the classic Heston pricing before introducing the Carr–Madan approach and its numerical considerations.

### 4.1 Fourier Transform Approach to Option Pricing

In Chapter 3 we said that the Heston model does not have a closed-form probability density function, which makes direct computation of option prices difficult, as integrals involving the density cannot be evaluated in closed form. However, since the characteristic function is known in closed form, we can use Fourier-based methods to price options.

The first approach using the Fourier transform to price options is Heston’s classical pricing formula [2], which uses the Gil-Pelaez inversion formula to obtain the risk-neutral probabilities from the characteristic functions. We have from Heston that the price of a European call option with strike  $K$  and maturity  $T$  is given by

$$C(S, v, t) = S_t F(t, T) P_1 - K P(t, T) P_2 \quad (4.1)$$

where  $F(t, T) = e^{-q\tau}$  is the forward factor, which accounts for the dividends,  $P(t, T) = e^{-r\tau}$  is the discount factor.  $P_1$  and  $P_2$  are the risk-neutral probabilities of the option expiring in the money. As stated these probabilities are obtained by using the Gil-Pelaez inversion formula

$$P_j = \mathbb{P}(\log S_T > \log K) = \frac{1}{2} + \frac{1}{\pi} \int_0^\infty \operatorname{Re} \left[ \frac{e^{-iu \log K} f_j(u; x, v)}{iu} \right] du \quad (4.2)$$

for  $j = 1, 2$ .

We now see that this approach obtains a closed form solution, but it should be noted that some refer to this as a semi closed form as numerical integration is required to compute  $P_j$ . This motivates the Carr–Madan framework, which we present in the following section.

### 4.2 The Carr–Madan Framework

Carr and Madan [3] introduced a method that expresses the option price directly as a Fourier transform of the characteristic function, which can be evaluated efficiently using



the FFT, which is an algorithm that reduces computational complexity from  $O(N^2)$  to  $O(N \log N)$ .

The idea is to work with the call price as a function of the log strike  $k = \log K$  instead of computing probabilities  $P_1$  and  $P_2$  separately. As shown in the previous section, Heston's original approach requires two separate numerical integrations for each strike price. Whereas, the Carr–Madan method computes option prices for all strike prices simultaneously by leveraging the FFT.

We denote the call price as a function of  $k$  by  $C_T(k)$ .

$$C_T(k) = \int_k^\infty e^{-r\tau} (e^s - e^k) q_T(s) ds \quad (4.3)$$

where  $q_T(s)$  is the risk-neutral density of the log price and  $s = \log S_T$ . We have that when  $k \rightarrow -\infty$  the call price converges to  $S_0$ , which means that  $C_T(k)$  is not in  $L^2$ , which is problematic for Fourier transform methods. A square-integrable function can be obtained by following Carr and Madan [3] and introducing the modified call price

$$c_T(k) = \exp(\alpha k) C_T(k) \quad (4.4)$$

where  $\alpha > 0$  is called the damping factor. The role of  $\alpha$  is to ensure that  $c_T(k) \in L^2$ , which allows the application of Fourier transform methods and ensures stable numerical inversion. The damping factor will be discussed in detail in Section 4.4.

The Fourier transform of  $c_T(k)$  is defined as

$$\psi_T(v) = \int_{-\infty}^\infty e^{ivk} c_T(k) dk \quad (4.5)$$

We can now substitute equation (4.4) into equation (4.5), which gives us

$$\psi_T(v) = \int_{-\infty}^\infty e^{ivk} e^{\alpha k} C_T(k) dk$$

We substitute the call price  $C_T(k) = e^{-r\tau} \int_k^\infty (e^s - e^k) q_T(s) ds$ , which gives us

$$\psi_T(v) = \int_{-\infty}^\infty e^{(\alpha+iv)k} e^{-r\tau} \int_k^\infty (e^s - e^k) q_T(s) ds dk$$

We now reverse the order of integration so that we integrate over  $k$  first. Since  $q_T(s)$  only contributes when  $k \leq s$ , the limits become

$$\psi_T(v) = e^{-r\tau} \int_{-\infty}^\infty q_T(s) \int_{-\infty}^s e^{(\alpha+iv)k} (e^s - e^k) dk ds$$

We split the inner integral into two parts

$$\int_{-\infty}^s e^{(\alpha+iv)k} e^s dk - \int_{-\infty}^s e^{(\alpha+iv)k} e^k dk$$

The first integral evaluates to  $\frac{e^{s(\alpha+1+iv)}}{\alpha+iv}$  and the second to  $\frac{e^{s(\alpha+1+iv)}}{\alpha+1+iv}$ , both of which converge because  $\alpha > 0$ . Combining these two results gives us

$$\psi_T(v) = e^{-r\tau} \int_{-\infty}^\infty q_T(s) e^{s(\alpha+1+iv)} \left[ \frac{1}{\alpha+iv} - \frac{1}{\alpha+1+iv} \right] ds$$

We combine the fractions, which gives us  $\frac{1}{\alpha+iv} - \frac{1}{\alpha+1+iv} = \frac{1}{(\alpha+iv)(\alpha+1+iv)}$ . We now recognise that the remaining integral is the characteristic function  $f_2(u) = \int_{-\infty}^{\infty} e^{ius} q_T(s) ds$  derived in Chapter 3 evaluated at  $u = v - (\alpha + 1)i$ , which gives us

$$\psi_T(v) = \frac{e^{-r\tau} f_2(v - (\alpha + 1)i)}{\alpha^2 + \alpha - v^2 + i(2\alpha + 1)v} \quad (4.6)$$

where we have expanded the denominator  $(\alpha + iv)(\alpha + 1 + iv) = \alpha^2 + \alpha - v^2 + i(2\alpha + 1)v$ .

The call price is then recovered by the inverse Fourier transform

$$C_T(k) = \frac{e^{-\alpha k}}{2\pi} \int_{-\infty}^{\infty} e^{-ivk} \psi_T(v) dv = \frac{e^{-\alpha k}}{\pi} \int_0^{\infty} \text{Re}[e^{-ivk} \psi_T(v)] dv \quad (4.7)$$

where the second equality follows from the fact that  $C_T(k)$  is real, which implies that the integrand has symmetric real and antisymmetric imaginary parts.

### 4.3 Fast Fourier Transform Algorithm

The integral in equation (4.7) can be approximated numerically using the trapezoidal rule. We can discretize the integration variable  $v$  with the step size  $\eta$ , so that  $v_j = \eta(j - 1)$  for  $j = 1, \dots, N$ , where  $N = 2^n$ . The log strike values are given by

$$k_m = -b + \lambda(m - 1), \quad m = 1, \dots, N \quad (4.8)$$

where  $\lambda$  is the step size of the log strikes, and  $b = \frac{N\lambda}{2}$  so that the log strikes range approximately from  $-b$  to  $b$ . The step size  $\eta$  and  $\lambda$  have a relation given as

$$\lambda\eta = \frac{2\pi}{N} \quad (4.9)$$

which shows that a finer grid in the Fourier variable  $v$  (small  $\eta$ ) results in a less refined grid in  $k$  (large  $\lambda$ ), and vice versa.

To improve the accuracy of the discretized integral, we apply Simpson's rule instead of the trapezoidal rule, as it uses a quadratic polynomial approximation between grid points, which gives a higher order of accuracy for smooth integrands. The call price at log strike  $k_m$  is approximated as

$$C_T(k_m) \approx \frac{e^{-\alpha k_m}}{\pi} \text{Re} \left[ \sum_{j=1}^N e^{-i\lambda\eta(j-1)(m-1)} e^{ibv_j} \psi_T(v_j) \eta w_j \right] \quad (4.10)$$

where we have that

$$w_1 = w_N = \frac{1}{3} \quad \text{and} \quad w_j = \begin{cases} \frac{4}{3}, & j \text{ even} \\ \frac{2}{3}, & j \text{ odd} \end{cases} \quad \text{for } j = 2, \dots, N - 1.$$

and we define the summation as

$$e_m = \text{Re} \left[ \sum_{j=1}^N e^{-i\lambda\eta(j-1)(m-1)} e^{ibv_j} \psi_T(v_j) \eta w_j \right]$$

The summation is in the exact form required by the FFT algorithm. This means that the FFT computes option prices for all  $N$  strike prices simultaneously, which is a significant improvement over the Gil-Pelaez approach in equation (4.2), where each strike requires a separate numerical integration.

## 4.4 Damping Factor and Numerical Considerations

The damping factor  $\alpha$  in equation (4.4) plays an important role in the accuracy and stability of the method. It must satisfy two conditions in order to ensure numerical stability. Firstly, we have that  $\alpha > 0$  to ensure that the modified call price is square-integrable. Secondly, the  $(1 + \alpha)$ -th moment of  $S_T$  must be finite, which requires that  $f_2(-(1 + \alpha)i)$  is finite [3]. This condition ensures that the integral  $\int_{-\infty}^{\infty} e^{(\alpha+1)s} q_T(s) ds$  converges, which is necessary for the derivation of  $\psi_T(v)$  in equation (4.6), as the inner integrals require exponential terms involving  $e^{(\alpha+1)s}$  to be integrable. We also have that  $\alpha$  plays a crucial role beyond the theoretical conditions. If  $\alpha$  is too small, the integrand decays slowly and the truncation error is large, which implies loss of accuracy in the FFT approximation. If  $\alpha$  is too large, the integrand becomes highly oscillatory, which causes numerical instability. In practice,  $\alpha$  is typically set in an interval of  $[1, 2]$  to provide a balance between the numerical stability and accuracy.

The choice of  $N$  and  $\eta$  also plays a role in how effective the method is in terms of accuracy. A larger  $N$  provides more strike prices and reduces the truncation error, but increases the computation time. The step size  $\eta$  controls the upper limit of integration  $N\eta$ , and it should be chosen large enough to capture the significant part of the integrand. Carr and Madan [3] recommend choosing  $N = 4096$  and  $\eta = 0.25$  to balance accuracy and computational efficiency.

# Chapter 5

## Implementation

In this chapter we implement the Carr–Madan method introduced in the previous chapter as well as discussing the validation, which includes the choice of parameters. We validate the implementation by comparing the FFT prices against Heston’s original pricing formula from Section 4.1, which will serve as a benchmark.

### 5.1 Implementation of the Carr–Madan Method

We implement the FFT algorithm introduced in Section 4.3 using Python, where we utilize the NumPy library for mathematical operations.

---

**Algorithm 1** Carr–Madan FFT Option Pricing

---

**Input:**  $N, \eta, \alpha, S, \tau, \kappa, \theta, \sigma, \rho, v_0, r, q$

**Output:** Arrays of strikes  $K$  and call prices  $C$

$\lambda \leftarrow 2\pi/(N\eta)$

$b \leftarrow N\lambda/2$

$s_0 \leftarrow \log S$

**for**  $j = 0, 1, \dots, N - 1$  **do**

$v_j \leftarrow j\eta$

$k_j \leftarrow -b + j\lambda + s_0$

$f_{2,j} \leftarrow f_2(v_j - (\alpha + 1)i)$

$\triangleright$  Heston characteristic function

$\psi_j \leftarrow \frac{e^{-r\tau} f_{2,j}}{\alpha^2 + \alpha - v_j^2 + i(2\alpha + 1)v_j}$

$w_j \leftarrow$  Simpson weight

$x_j \leftarrow e^{i(b-s_0)v_j} \psi_j w_j$

**end for**

$\mathbf{e} \leftarrow \text{FFT}(\mathbf{x})$

**for**  $j = 0, 1, \dots, N - 1$  **do**

$C_j \leftarrow \frac{\eta}{\pi} e^{-\alpha k_j} \text{Re}(e_j)$

$K_j \leftarrow e^{k_j}$

**end for**

---

The pseudo code of the implementation can be seen above. Note that we implement the algorithm in a vectorized form using NumPy [8], which helps us avoid loops and therefore improves computational efficiency. Also note that following Rouah [7], we shift

the log-strike grid by adding  $s_0 = \log S$  to  $k_j$ . This centers the FFT grid around the ATM region ( $K \approx S_0$ ), which improves numerical accuracy for the most relevant strikes. The Heston characteristic function is used as defined in Chapter 3. The Fast Fourier Transform is applied using `np.fft.fft` from NumPy.

## 5.2 Validation and Numerical Stability

We validate the implementation of the algorithm by comparing the results produced against those given by Rouah [7, p. 140]. By setting the parameters to  $N = 1024$ ,  $\eta = 0.0977$ ,  $\alpha = 1.5$ ,  $S = 50$ ,  $\tau = 0.5$ ,  $\kappa = 0.2$ ,  $\theta = 0.05$ ,  $\sigma = 0.3$ ,  $\rho = -0.7$ ,  $v_0 = 0.05$ ,  $r = 0.03$ ,  $q = 0.05$ , we obtain the results shown in the table below. The small absolute errors indicate strong validation of the algorithm implementation.

| Strike (Rouah) | Strike (FFT) | Price (Rouah) | Price (FFT) | Absolute Error |
|----------------|--------------|---------------|-------------|----------------|
| 41.4102        | 41.4137      | 8.6378        | 8.6352      | 0.0026         |
| 44.0956        | 44.0981      | 6.4765        | 6.4742      | 0.0023         |
| 46.9551        | 46.9564      | 4.4453        | 4.4445      | 0.0008         |
| 50.0000        | 50.0000      | 2.6778        | 2.6782      | 0.0004         |
| 53.2424        | 53.2409      | 1.3269        | 1.3272      | 0.0003         |
| 56.6950        | 56.6918      | 0.5020        | 0.5023      | 0.0003         |
| 60.3716        | 60.3665      | 0.1421        | 0.1427      | 0.0006         |

Table 5.1: Comparison of FFT Call Prices with Calculations by Rouah

We implement the reformulation known as the Heston trap introduced by Albrecher [9], which addresses a numerical instability in the original characteristic function. The issue arises because the complex logarithm in  $C_j$  in Equation 3.3 can produce discontinuities when the argument of the logarithm crosses a branch cut, which leads to incorrect option prices. The Heston trap reformulates the characteristic function by replacing  $g_j$  with  $1/g_j$ , which ensures that the argument of the logarithm remains in the principal branch and avoids the discontinuity.

As discussed in Section 4.4, the choice of  $\alpha$ ,  $N$ , and  $\eta$  affects the numerical stability of the algorithm. Therefore, we run a test to pick the parameters, where the results plotted in the figure below contain three plots that show absolute error against explicit Heston option price with different parameter choices under different option scenarios.

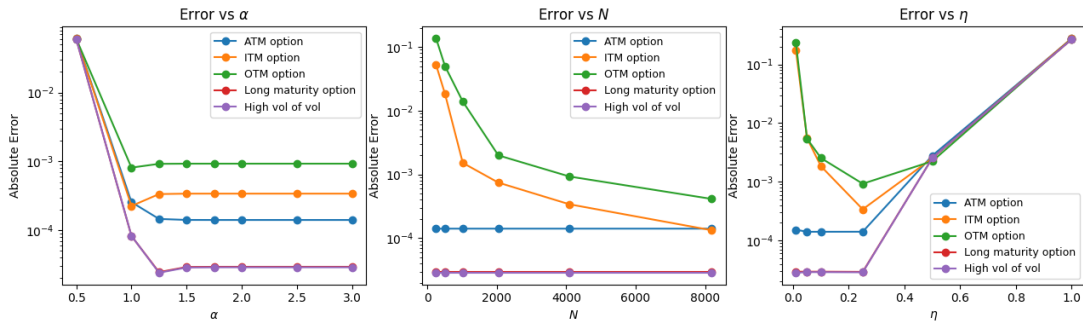


Figure 5.1: Plots showing absolute pricing error as a function of model parameters

We see from the first plot that the graph seems to be consistent across the different scenarios, where the trend is that increasing  $\alpha$  leads to a decreasing absolute error until  $\alpha = 1.5$ , afterwards it becomes stable.

The second plot shows us a consistent trend across all scenarios as well, and we can see that the error is decreasing as  $N$  increases, which is consistent with the theory introduced in Section 4.4. We also notice that the significant fall in absolute error stops happening beyond  $N = 4096$ .

Finally, the third plot shows us that  $\eta = 0.25$  is the best performing value in terms of the absolute error. This also aligns with the conclusion we make based on Carr–Madan in Section 4.4.

Based on the parameter analysis, we set our parameters to  $\alpha = 1.5$ ,  $N = 4096$  and  $\eta = 0.25$  for the data generation for Chapter 6.

Generally, we see a trend in which options can be the hardest numerically to price. This trend is as follows for the absolute error  $\delta$

$$\delta_{\text{OTM}} > \delta_{\text{ITM}} > \delta_{\text{ATM}},$$

which should be interpreted as OTM being the hardest to price where ATM options tend to be the easiest to price of the three.

# Chapter 6

## Neural Network Acceleration

In this chapter, we introduce the main topic of this thesis, where we develop a neural network that approximates the Heston pricing function. The idea is to train a neural network on option prices generated by the Carr–Madan FFT method from Chapter 4, so that the trained network can compute option prices significantly faster than the FFT, and we do not need to do the integration. We first motivate the use of neural networks, then describe the network architecture, training data, training procedure, and the loss function used for optimization.

### 6.1 Motivation for Neural Network Surrogates

The Carr–Madan FFT method presented in Chapter 4 computes option prices efficiently for a grid of strikes given a fixed set of Heston parameters. However, it has two limitations which have already been discussed in Section 1.1. The two limitations being each new set of parameters requires a new FFT computation and interpolation is needed if the desired strike is not on the strike grid.

Neural networks offer a possible solution to both of these limitations. A neural network is a function approximator that can learn to map inputs to outputs from data [10]. A key theoretical result that supports the use of neural networks for this purpose is the universal approximation theorem, which states that a feedforward neural network with a single hidden layer containing a sufficient number of neurons can approximate any continuous function on a compact subset of  $\mathbb{R}^n$  to arbitrary precision [11]. This means that the Heston pricing function can be approximated by a neural network, though in practice a deeper network with multiple hidden layers is needed.

The use of neural networks for derivative pricing was first explored by Hutchinson, Lo and Poggio [12], who showed that neural networks could learn the Black–Scholes pricing function from simulated data. Liu, Oosterlee and Bohte [13] extended this to the Heston model, training neural networks on prices generated by the COS method, which is an alternative Fourier-based pricing method to the Carr–Madan FFT used in this thesis. Horvath, Muguruza and Tomas [14] trained neural networks directly on implied volatility surfaces under rough volatility models, demonstrating that neural networks can learn the mapping from model parameters to implied volatilities. Our approach builds on these ideas by training a surrogate model using prices generated by the Carr–Madan FFT method, and additionally training a second network directly on implied volatilities.

## 6.2 Network Architecture

We use a feedforward neural network, which is also known as a multilayer perceptron (MLP) [10]. A feedforward neural network consists of an input layer, one or more hidden layers, and an output layer. The information goes in one direction, from the input layer through the hidden layers to the output layer.

An input layer is the layer in which we feed our input parameters where each parameter corresponds to a neuron. A neuron, also called perceptron is a unit, which can receive input, process it and produce an output. The term "network" is essentially referring to multiple neurons, which are interconnected and form a network for processing the information that flows through the layers. The hidden layers refer to the layers whose values are not directly observed in the data, but are instead learned during training. These layers perform computations that transform the input into a form from which the output layer can produce the final prediction. The output layer is the final layer of the network, which produces the predicted value, in our case this is the option price.

Each layer performs an affine transformation followed by an element-wise nonlinear activation function. The output of the  $k$ -th hidden layer is given by

$$\mathbf{h}^k = \phi(\mathbf{b}^k + \mathbf{W}^k \mathbf{h}^{k-1}), \quad k = 1, \dots, L \quad (6.1)$$

where  $\mathbf{h}^0 = \mathbf{x}$  is the input,  $\mathbf{W}^k$  is the weight matrix which affects how much one neuron influences another,  $\mathbf{b}^k$  is the bias vector which allows each neuron to learn a shift, and  $\phi$  is the activation function applied element-wise [10, p. 162]. The activation function introduces nonlinearity, which is what allows the network to approximate complex nonlinear functions such as the Heston pricing function.

The final output of the network is computed by the output layer as

$$\hat{y} = \mathbf{w}^{L+1} \cdot \mathbf{h}^L + b^{L+1} \quad (6.2)$$

where  $\mathbf{w}^{L+1}$  is the weight vector and  $b^{L+1}$  is the bias of the output layer. We note that the output layer does not have an activation function, which allows the network to output any real value, as option prices are continuous and non-negative.

In our model, we use the Rectified Linear Unit (ReLU) as the activation function for all hidden layers, which is defined as

$$\phi(a) = \max(0, a) \quad (6.3)$$

which shows that if the input is below 0, the ReLU activation function returns 0 which indicates that some neurons are inactive. ReLU is a common choice in practice, as it helps to avoid the problem of gradients becoming too small. The small gradients are a problem, as the network uses gradients to update the parameters and improve the model.

Our network has an architecture with 9 neurons for the input layer, which correspond to the 9 input features log-moneyness  $\log(\frac{K}{S})$ , time to maturity  $\tau$ , mean reversion speed  $\kappa$ , long-run mean of variance  $\theta$ , volatility of volatility  $\sigma$ , correlation  $\rho$ , initial variance  $v_0$ , risk-free rate  $r$ , and dividend yield  $q$ . Note that we use log-moneyness to make the learning for the network easier, as it becomes scale invariant. The network has 4 hidden layers ( $L = 4$ ) where each layer has 128 neurons, and a single output neuron that gives the predicted option price. A visualization of the architecture can be seen below, where



each node corresponds to a neuron and each line is a connection between the neurons in different layers.

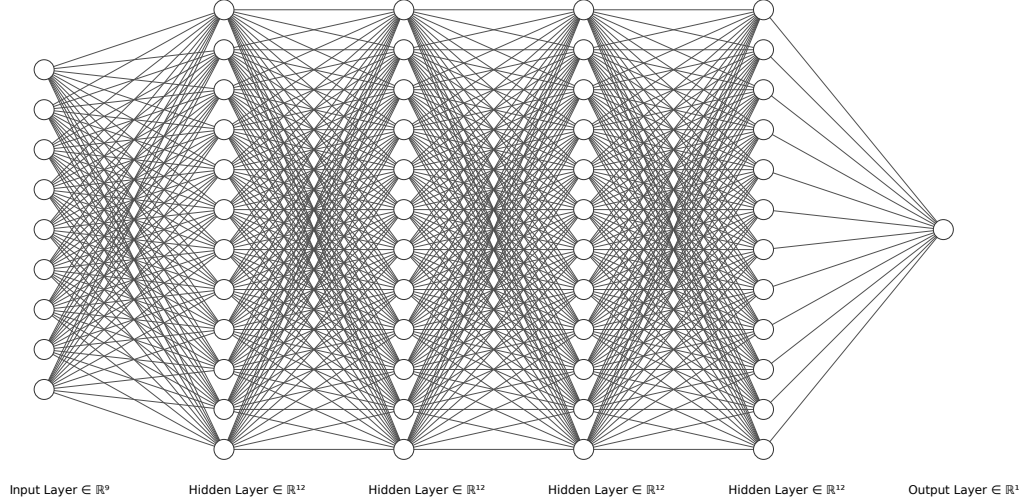


Figure 6.1: Architecture of the MLP used for Heston model approximation

Note that in the figure we only visualize 12 neurons in the hidden layer due to clarity of the figure. In practice, this should be interpreted as 128 neurons in each hidden layer. The final architecture is a fully connected network as follows

$$9 \rightarrow 128 \rightarrow 128 \rightarrow 128 \rightarrow 128 \rightarrow 1$$

### 6.2.1 Architecture Selection

To find the architecture in the previous section, we run tests on different architectures with fixed parameters, where the parameter selection will be discussed later. We test 5 different architectures, where each architecture is tested on 3 different seeds to evaluate consistency. The architectures tested are  $3 \times 128$ ,  $4 \times 64$ ,  $4 \times 128$ ,  $4 \times 256$ , and  $5 \times 128$ , where the notation  $L \times i$  refers to  $L$  hidden layers with  $i$  neurons each. The averaged results across the 3 seeds can be seen in the table below.

Note that throughout this section, the absolute error is represented for  $S = 100$ .

We see that the  $4 \times 256$  architecture achieves the best results in terms of mean relative error (0.12%), mean absolute error, and validation loss. However, it comes at a significant cost in training time as it takes 87.6 minutes compared to 76.9 minutes for the  $4 \times 128$  architecture. The  $4 \times 128$  architecture achieves a mean relative error of 0.14%, which is close to the result of  $4 \times 256$  architecture, while having less than a third of the parameters and training approximately 12% faster.

We also note that the  $3 \times 128$  architecture performs similarly to the  $4 \times 128$  in terms of mean relative error, but the  $4 \times 128$  has a lower validation loss on average. The  $4 \times 64$  architecture performs the worst, which indicates that 64 neurons per layer is insufficient to capture the complexity of the Heston pricing function. The  $5 \times 128$  architecture performs

| Metric                                  | $3 \times 128$ | $4 \times 64$ | $4 \times 128$ | $4 \times 256$ | $5 \times 128$ |
|-----------------------------------------|----------------|---------------|----------------|----------------|----------------|
| Parameters                              | 34,433         | 13,185        | 50,945         | 200,193        | 67,457         |
| Mean relative error (%)                 | 0.15           | 0.22          | 0.14           | <b>0.12</b>    | 0.14           |
| Median relative error (%)               | 0.04           | 0.06          | 0.04           | <b>0.03</b>    | 0.04           |
| Mean absolute error ( $S = 100$ )       | 0.00979        | 0.01460       | 0.00935        | <b>0.00836</b> | 0.00930        |
| Best validation loss                    | 1.35e-6        | 2.64e-6       | 1.17e-6        | <b>1.01e-6</b> | 1.30e-6        |
| MRE standard deviation across seeds (%) | 0.007          | 0.008         | <b>0.002</b>   | 0.003          | 0.004          |
| Training time (min)                     | <b>70.3</b>    | 70.3          | 76.9           | 87.6           | 81.1           |

Table 6.1: Average model performance across different architectures

comparably to the  $4 \times 128$ , with both achieving a mean relative error of 0.14%, though the  $5 \times 128$  has more parameters and longer training time.

We see from the standard deviation of the mean relative error across seeds that the  $4 \times 128$  architecture is the most consistent with a standard deviation of 0.002%, while the  $4 \times 64$  shows the highest variance at 0.008%. This indicates that wider layers improve consistency.

Based on these results, we selected the  $4 \times 128$  architecture in the previous section, as it provides a good balance between accuracy, consistency, and training time. The detailed results of Table 6.1 can be seen in Appendix B.1.2

## 6.3 Training Data

The training data is generated using the Carr–Madan FFT implementation from Chapter 5 with the parameters  $\alpha = 1.5$ ,  $N = 4096$ , and  $\eta = 0.25$  as determined in Section 5.2. We define a function `def random_parameters()`, which utilizes NumPy to sample parameters from the ranges shown in the table below.

| Parameter | Range        | Distribution |
|-----------|--------------|--------------|
| $\tau$    | [0.01, 2]    | Log-uniform  |
| $\kappa$  | [0.5, 5]     | Uniform      |
| $\theta$  | [0.01, 0.16] | Uniform      |
| $\sigma$  | [0.1, 0.8]   | Uniform      |
| $\rho$    | [−0.9, 0.0]  | Uniform      |
| $v_0$     | [0.01, 0.16] | Uniform      |
| $r$       | [0.01, 0.05] | Uniform      |
| $q$       | [0, 0.05]    | Uniform      |

Table 6.2: Parameter ranges for training data generation

As we previously discussed in Chapter 2, the model has an assumption that the variance must be positive. We therefore introduced the Feller condition 2.1, which we also implement when running the data generation to ensure positive variance dynamics and numerically stable training data. This means that the training distribution is not an independent uniform distribution over all parameter ranges, but is conditional on the Feller condition being satisfied.

We generate 20,000 random parameter sets, and for each set we compute option prices across the FFT strike grid. We then filter the results to only include data points where the log-moneyness  $\log(K/S)$  is in the range  $[-0.5, 0.5]$  and the option price is in the range  $[0.0001, 0.5]$ . Note that we fix  $S = 1$ , so the option prices are not expressed in absolute currency units, which helps the model to be scale invariant. After the filtering, we have 2582580 training samples in total.

We notice that the time to maturity  $\tau$  is sampled from a log-uniform distribution rather than a uniform distribution. This is because we want the network to see more short maturity options, which are more common in practice and where the pricing function varies more rapidly. A uniform distribution over  $[0.01, 2]$  would oversample long maturity options, where the pricing function is smoother and easier to learn.

Before training, we standardize all input features and the target variable to have zero mean and unit variance using the `StandardScaler` from scikit-learn [15]. If we did not do the standardization, the features with larger magnitudes would dominate the learning process, which would slow down convergence and reduce accuracy [10, Section 8.2].

## 6.4 Training Procedure

We begin the training procedure by splitting the data into a training set, which corresponds to 80% of the data, and validation and test sets, each containing half of the remaining data. To prevent data leakage, we use `GroupShuffleSplit` from scikit-learn [15], which ensures that all strike prices belonging to the same set of Heston parameters are assigned to the same split. The training is performed on an NVIDIA Quadro P2000 with Max-Q Design GPU using PyTorch [16]. The training set is used to update the network weights, the validation data is used during training for early stopping, which will be discussed later in this section, and the test set is reserved to evaluate the performance of the trained network on unseen data.

The network is trained using mini-batch gradient descent, where we evaluate batch sizes of 1024, 2048, 4096, 8192 and 16384 to find the best performing batch size. In each epoch, the training data is divided into batches, and for each batch the network computes predictions through a forward pass, the loss is computed, and the gradients of the loss with respect to the network parameters  $\theta = \{\mathbf{W}^k, \mathbf{b}^k\}_{k=1}^{L+1}$  are computed using backpropagation [10].

The network weights are initialized using PyTorch’s [16] default initialization, which is based on Kaiming initialization [17]. This method draws the weights from a uniform distribution scaled by the number of input neurons in each layer, which is designed to maintain stable gradients across layers when using ReLU activation functions.

To prevent overfitting, we implement early stopping. After each epoch, we evaluate the model on the validation set and compare the validation loss to the best validation loss observed so far. If the validation loss does not improve for 25 consecutive epochs, we stop the training and load the model with the best validation loss, which follows the patience based early stopping approach described by Prechelt [18]. This ensures that we use the model that generalizes best to unseen data rather than the model from the last epoch.

The Adam optimizer [19] is used to update the weights, which is a variant of stochastic gradient descent that adapts the learning rate for each parameter individually. The optimizer computes adaptive learning rates based on estimates of the first and second

moments of the gradients, which makes it converge faster and more robust to the choice of the initial learning rate compared to standard gradient descent.

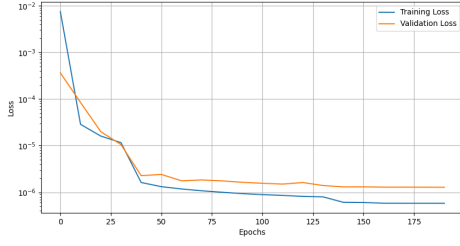
We set the initial learning rate to 0.001 and apply a learning rate scheduler that uses the validation loss to adjust the learning rate. If the validation loss does not improve for 10 consecutive epochs, the scheduler reduces the learning rate by a factor of 0.1. This allows the network to take larger steps at the beginning of the training and smaller steps as the validation loss begins to reach a minimum, which helps to improve the final accuracy.

We set the batch size to 1024 after evaluating the performance of the model on different batch sizes. The observed results can be seen in the table below.

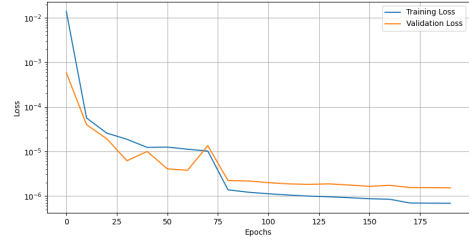
| <b>Metric</b>             | <b>1024</b>    | <b>2048</b> | <b>4096</b>   | <b>8192</b> | <b>16384</b> |
|---------------------------|----------------|-------------|---------------|-------------|--------------|
| Mean relative error (%)   | <b>0.15</b>    | 0.16        | 0.18          | 0.22        | 0.35         |
| Median relative error (%) | <b>0.04</b>    | <b>0.04</b> | 0.05          | 0.07        | 0.11         |
| Max relative error (%)    | 10.25          | <b>9.46</b> | 13.76         | 15.30       | 18.51        |
| Mean absolute error       | <b>0.0098</b>  | 0.0104      | 0.0124        | 0.0154      | 0.0239       |
| Median absolute error     | <b>0.0062</b>  | 0.0068      | 0.0084        | 0.0109      | 0.0176       |
| Max absolute error        | 0.2255         | 0.2224      | <b>0.2124</b> | 0.2602      | 0.3425       |
| Final training loss       | <b>5.83e-7</b> | 6.87e-7     | 1.08e-6       | 2.05e-6     | 5.24e-6      |
| Final validation loss     | <b>1.28e-6</b> | 1.52e-6     | 1.95e-6       | 3.00e-6     | 6.17e-6      |

Table 6.3: Model performance across different batch sizes

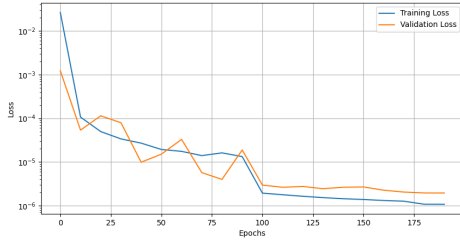
We see that a batch size of 1024 has the best results across all metrics except the max relative error, which is lower for batch size 2048, and the max absolute error, which is lower for batch size 4096. However, this does not change the overall image that we see, which shows that batch size of 1024 is performing better overall for our neural network. This behavior is expected, as larger batch sizes result in fewer parameter updates per epoch, which gives the model less opportunity to refine its weights. We also note that the training time does not vary significantly across batch sizes, which indicates no computational trade off that justifies larger batch size in our case. The full training logs for each batch size can be seen in Appendix [B.1.3](#).



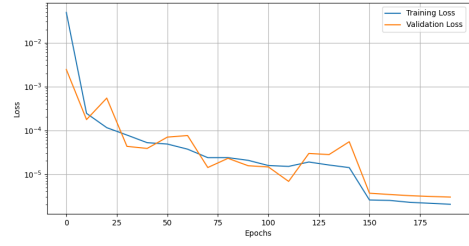
(a) Batch size 1024



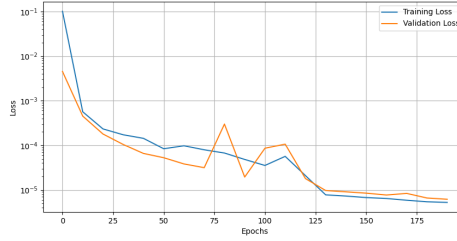
(b) Batch size 2048



(c) Batch size 4096



(d) Batch size 8192



(e) Batch size 16384

Figure 6.2: Training and validation loss curves for different batch sizes

In the figure above, we have the training and validation loss curves for each batch size. We see that smaller batch sizes converge to lower loss values, with batch size of 1024 reaching the lowest final loss. The validation loss remains consistently above the training loss across all batch sizes, which is expected as the model has not been trained on the validation data. The validation loss curves also show some fluctuation across all batch sizes, which indicates some instability in generalization. For the smaller batch sizes, a sharp drop in training loss is visible when the learning rate scheduler reduces the learning rate. We also observe that all batch sizes have converged within the 200 epoch limit, which indicates that 200 epochs is sufficient for the network to reach stable loss values despite the fluctuation in validation loss. These results are consistent with what we saw in Table 6.3.

## 6.5 Loss Function and Optimisation

The loss function measures the difference between the predicted option prices  $\hat{y}_i$  and the true option prices  $y_i$  generated by the FFT. We use mean squared error as the loss function,

which is defined as

$$\mathcal{L}(\boldsymbol{\theta}) = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2 \quad (6.4)$$

where  $n$  is the number of training samples. MSE is a standard choice when it comes to regression problems. The reason being that it is well defined for mathematical operations such as differentiability, which makes it a good choice when it comes to optimization. Another reason is that it penalizes large errors more heavily than small errors due to the squaring, which makes it useful when we want the network to avoid large pricing errors [10].

The goal of the training is to find parameters  $\boldsymbol{\theta}$  that minimize the loss function. This optimization problem can be written as

$$\boldsymbol{\theta}^* = \arg \min_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}) \quad (6.5)$$

where  $\boldsymbol{\theta}^*$  denotes the optimal parameters. Note that we do not have a guarantee that a global optimum can be found during training, as neural networks generally produce a non convex optimization problem. This means that in practice when we find a solution it might be a local minimum rather than the global minimum, or even a saddle point. Nevertheless, empirical evidence suggests that for sufficiently large networks, most local minima tend to have comparable performance, which indicates that finding a good local minimum is often sufficient in practice [20].

The performance of the trained neural network is evaluated on the test set using two primary error metrics. We compute the mean absolute error and mean relative error given by

$$\text{MAE} = \frac{1}{N} \sum |\hat{y}_i - y_i| \quad (6.6)$$

$$\text{MRE} = \frac{1}{N} \sum \frac{|\hat{y}_i - y_i|}{y_i} \quad (6.7)$$

where MAE measures the absolute pricing error and MRE gives us a measure of the pricing accuracy that is independent of the scale. The results of the neural network performance will be presented in Chapter 7.

# Chapter 7

## Results

In this chapter, we present the results produced by the Carr–Madan FFT algorithm from Section 5.1 and the results from our trained neural network defined in the previous chapter. We will then use these results to do a comparison and benchmarking of the two approaches in the form of pricing accuracy, computation speed, and generalization.

### 7.1 Carr–Madan Pricing Results

We use the parameter values in Section 5.2 and plot the option price produced by our Carr–Madan FFT algorithm against the benchmark Heston price, which is calculated using Equation 4.2.

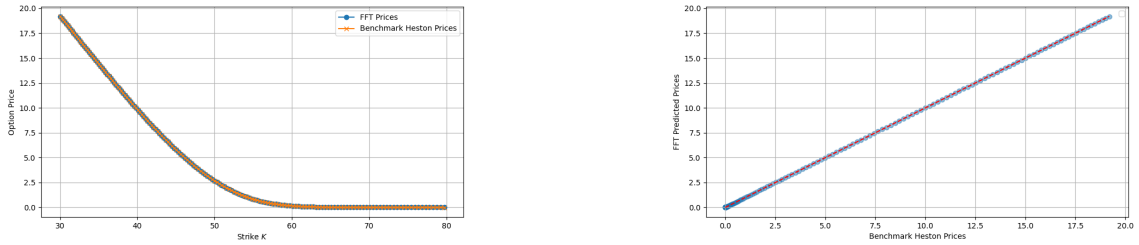


Figure 7.1: Carr–Madan FFT call option prices compared with benchmark Heston prices across strikes

We see from the left plot that both curves lie on top of each other, which indicates that the Carr–Madan FFT algorithm is accurately computing the option prices. This is also supported by the scatter plot, which shows that prices produced by the Carr–Madan FFT lie on the 45 degree line  $y = x$ , which supports the accuracy of our implemented algorithm.

By using the same parameter values across  $\rho = -0.8, -0.4$  and  $0$ , and  $\sigma = 0.1, 0.3$  and  $0.6$ , we produce the implied volatility curves introduced in Subsection 1.3.2, which will be used as a reference for the neural network performance.

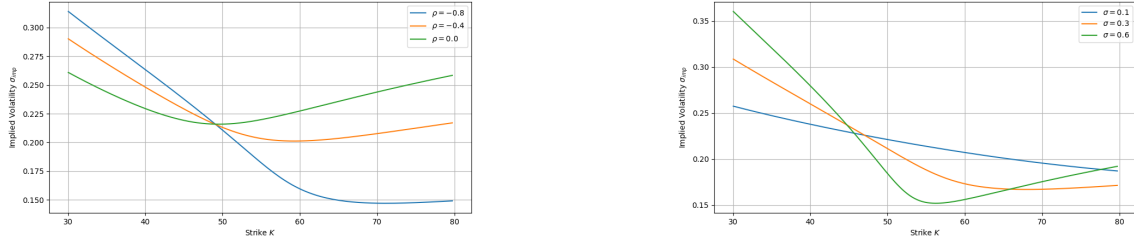


Figure 7.2: Implied volatility smiles produced by the Carr–Madan FFT for different values of  $\rho$  (left) and  $\sigma$  (right)

In the left plot, we see that a more negative  $\rho$  produces a steeper skew in the implied volatility smile, where ITM call options have higher implied volatility and OTM call options have lower implied volatility. We also notice that when  $\rho = 0$ , the smile becomes symmetric. This is consistent with the leverage effect discussed in Section 2.3, where a negative  $\rho$  implies that a drop in the asset price is associated with an increase in variance. In the right plot, we see that increasing  $\sigma$  produces a more curved smile with higher implied volatility for deep ITM and deep OTM options. A small value of  $\sigma = 0.1$  produces a nearly flat smile, which is close to the constant volatility assumption in the Black–Scholes model. This is consistent with the kurtosis effect discussed in Section 2.3.

## 7.2 Neural Network Performance

To evaluate the networks performance we use 5 parameter test cases, which are shown in the table below. The underlying asset price is fixed at  $S_0 = 50$  for all cases.

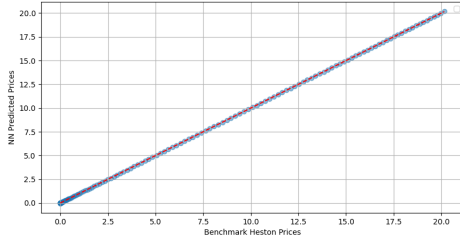
| Case | Description                  | $\tau$ | $\kappa$ | $\theta$ | $\sigma$ | $\rho$ | $v_0$ | $r$  | $q$  |
|------|------------------------------|--------|----------|----------|----------|--------|-------|------|------|
| 1    | Baseline benchmark           | 0.50   | 2.00     | 0.04     | 0.30     | -0.70  | 0.04  | 0.03 | 0.01 |
| 2    | Short maturity/high variance | 0.25   | 3.00     | 0.06     | 0.40     | -0.50  | 0.06  | 0.02 | 0.02 |
| 3    | Long maturity/low vol-of-vol | 1.00   | 1.50     | 0.03     | 0.20     | -0.30  | 0.03  | 0.04 | 0.01 |
| 4    | Extreme short maturity skew  | 0.10   | 4.00     | 0.08     | 0.50     | -0.80  | 0.08  | 0.03 | 0.00 |
| 5    | Long maturity/high variance  | 1.50   | 1.00     | 0.10     | 0.30     | -0.40  | 0.10  | 0.02 | 0.03 |

Table 7.1: Heston parameter test cases used to evaluate neural network pricing performance

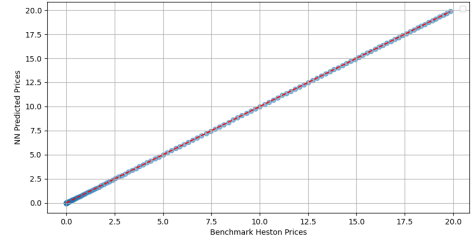
### 7.2.1 Pricing Accuracy

We begin by comparing the neural network predicted prices against the Carr–Madan FFT, which represents the benchmark prices in this comparison, using scatter plots where each point represents a strike price and the dashed line is  $y = x$ . The scatter plots for all five test cases can be seen below.

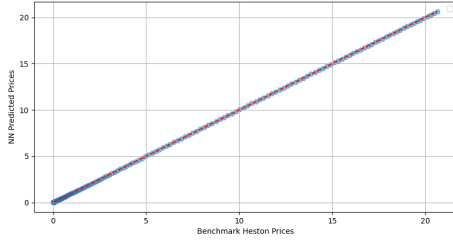




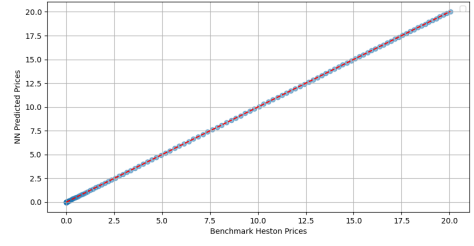
(a) Baseline



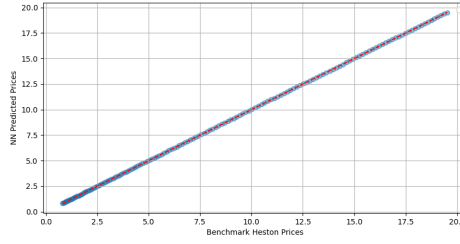
(b) Short/high variance



(c) Long/low vol-of-vol



(d) Extreme skew



(e) Long/high variance

Figure 7.3: Neural network predicted prices against Carr–Madan FFT benchmark prices across five representative Heston parameter cases

We see from the scatter plots that the neural network predictions lie close to the  $y = x$  line for all the cases, which indicates that the network has successfully learned the Heston pricing function. However, there are some deviations from the line for low priced options, which correspond to the OTM region. We examine this by plotting on a logarithmic scale in the appendix Figure B.2. We observe that the predictions for options with prices below approximately  $10^{-2}$  deviate significantly from the  $y = x$  line, particularly for the baseline, short maturity, and extreme skew cases. However, we note that the long maturity cases do not exhibit this behavior to the same degree, where the long maturity/high variance case in particular shows predictions that lie closely on the  $y = x$  line even for low-priced options. This can be explained by the fact that for these particular parameter combinations, the longer maturity allows the variance process more time to revert towards its long-run mean  $\theta$ , which reduces the sensitivity of the option price to small changes in the input parameters. Furthermore, a higher variance level increases the absolute price of OTM options, which means that the MSE loss function assigns more weight to these options during training. The deviations observed in the short maturity and extreme skew

cases are expected, as the pricing function varies most rapidly in the deep OTM region for short maturities, which makes it inherently more difficult to approximate.

As an additional check, we plot the option price as a function of strike  $K$ , where the figures are shown in appendix Figure B.1. We see the curves lie on top of each other, which shows that the neural network accurately reproduces the pricing function across the full range of strike prices. The logarithmic scale plots in appendix Figure B.3 show the same behavior as the scatter plots, where the deviations become visible for deep OTM options with very low prices.

The error metrics of the test cases in Table 7.1 are presented in the table below, where we compute the mean of each metric across all five parameter sets, grouped by moneyness. The detailed metrics can be seen in Appendix B.1.5.

| Metric    | OTM    | Near ATM      | ITM           |
|-----------|--------|---------------|---------------|
| MAE       | 0.0074 | 0.0074        | <b>0.0063</b> |
| Median AE | 0.0039 | 0.0073        | <b>0.0051</b> |
| RMSE      | 0.0108 | 0.0085        | <b>0.0079</b> |
| MRE (%)   | 8.49   | 0.24          | <b>0.06</b>   |
| Max AE    | 0.0284 | <b>0.0145</b> | 0.0181        |

Table 7.2: Neural network pricing errors averaged across five Heston parameter sets, grouped by moneyness

Note that for this section all absolute pricing errors are represented for  $S = 50$  and near ATM region is defined as strikes within 5% of the spot price. The predictions are rescaled by multiplying the normalized output by  $S$ .

We see from the table that the network performs best for ITM options, with a mean absolute error of 0.0063 and a mean relative error of 0.06%. The OTM region has the highest mean relative error at 8.49%, which is significantly larger than the other two regions and is driven by the small absolute prices of OTM options, which means that even small absolute errors result in large relative errors when divided by the true price in Equation 6.7. We note that the mean absolute error for OTM options (0.0074) is comparable to the near ATM region (0.0074), which shows that the network produces similar absolute pricing errors across both regions.

The ordering of absolute errors is  $\delta_{\text{OTM}} \approx \delta_{\text{Near ATM}} > \delta_{\text{ITM}}$ , which is broadly consistent with the FFT ordering in Section 5.2 where  $\delta_{\text{OTM}} > \delta_{\text{ITM}} > \delta_{\text{ATM}}$ , though the neural network shows a smaller gap between the OTM and near ATM regions. The ITM region is the easiest for the network to learn, which is expected as ITM option prices are dominated by intrinsic value and therefore vary more smoothly with respect to the input parameters, whereas the OTM and near ATM regions exhibit higher sensitivity and curvature in the pricing surface.

To verify that these results are not only driven by the five selected parameter cases, we also evaluate the neural network on the full held-out test set.

| Metric    | OTM    | Near ATM | ITM           |
|-----------|--------|----------|---------------|
| MAE       | 0.0052 | 0.0061   | <b>0.0041</b> |
| Median AE | 0.0031 | 0.0038   | <b>0.0026</b> |
| RMSE      | 0.0085 | 0.0092   | <b>0.0062</b> |
| MRE (%)   | 2.99   | 0.47     | <b>0.04</b>   |
| 95% AE    | 0.0172 | 0.0181   | <b>0.0136</b> |
| 99% AE    | 0.0310 | 0.0369   | <b>0.0226</b> |
| Max AE    | 0.0722 | 0.0482   | <b>0.0411</b> |

Table 7.3: Held-out pricing errors grouped by moneyness region

We see the same pattern as before. ITM options have the lowest absolute and relative errors, while OTM options have the highest relative error. This is again mainly due to the low absolute prices of OTM options, where small pricing errors become large when measured in relative terms. Overall, the held-out test results support that the network generalizes well within the sampled Heston parameter range.

## 7.2.2 Generalization Across Parameter Regions

To evaluate how well the neural network behaves across different parameters we compute the absolute error of the neural network by using Carr–Madan FFT as the benchmark and plot a heatmap, which can be seen below. We focus on  $\rho$ ,  $\sigma$ , and  $\tau$  as these are the parameters that have the most significant effect on the shape of the pricing surface, as discussed in Section 2.3.

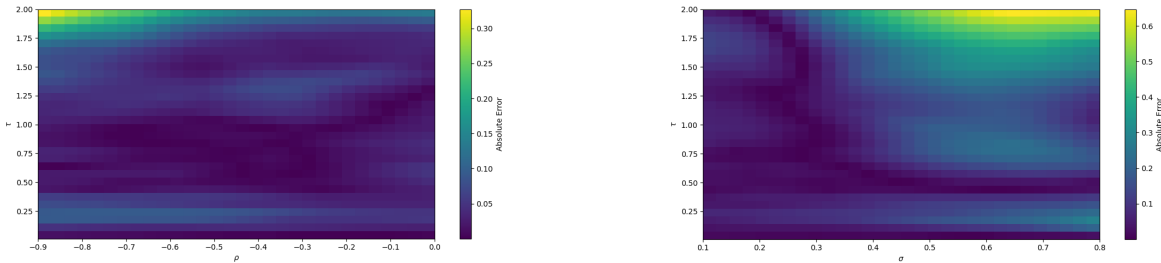


Figure 7.4: Heat map of the absolute error of the neural network relative to the FFT for different values of  $\rho$  vs  $\tau$  (left) and  $\sigma$  vs  $\tau$  (right)

We see from the left plot that the network has the lowest errors for  $\tau \in [0.40, 0.75]$  across all values of  $\rho$ . Very short maturities  $\tau < 0.40$  also exhibit noticeable errors due to the rapid variation of the pricing function. The highest errors appear for long maturities, exceeding 0.3 for  $\tau > 1.75$ . The error pattern is dominated by  $\tau$ , with clear horizontal banding showing that maturity has a stronger effect than correlation, where the long maturity errors can be explained by the loguniform sampling of  $\tau$  during training.

The right plot shows that the error increases with both  $\sigma$  and  $\tau$ , particularly for  $\sigma > 0.5$  combined with  $\tau > 1.5$ , where it exceeds 0.5. Short maturities  $\tau < 0.40$  also show a visible band of error. The error scale is larger in the right plot, which indicates that  $\sigma$  has a stronger effect on the network’s accuracy than  $\rho$ .

### 7.2.3 Implied Volatility Analysis

We evaluate the networks capability further by producing implied volatility smiles using the neural network and compare them against those produced by the Carr–Madan FFT in Section 7.1.

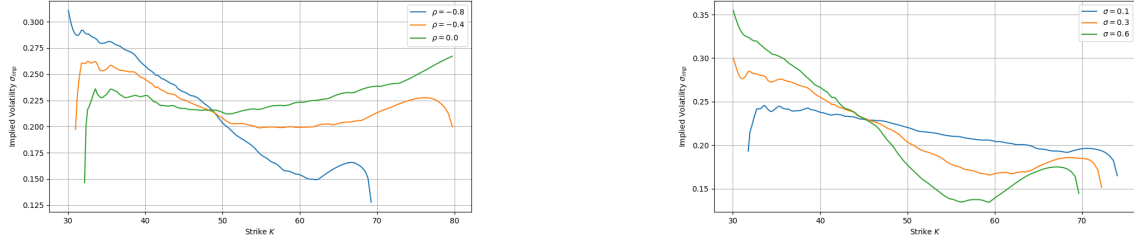


Figure 7.5: Implied volatility smiles produced by the neural network for different values of  $\rho$  (left) and  $\sigma$  (right)

We see from the figure that the neural network captures the shape of the implied volatility smiles produced by the FFT in Section 7.1, but shows some irregularities as the curves are not smooth. These irregularities can particularly be seen in the deep OTM region. This is because the network was trained to minimise the mean squared error on option prices, and not on implied volatilities directly. The small absolute pricing errors in the OTM region can lead to larger errors when converted to implied volatility, as the mapping from option price to implied volatility is highly nonlinear for low priced options.

We address this issue by training a second neural network directly on implied volatilities rather than option price. The architecture and training procedure remain the same as described in Section 6.2.1 and Section 6.4, with the learning rate reduction factor set to 0.5 instead of 0.1. The training log and performance metrics for this network can be seen in Appendix B.1.4.

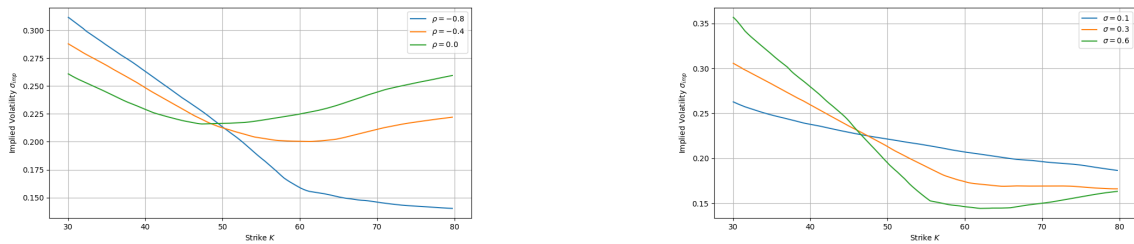


Figure 7.6: Implied volatility smiles produced by the implied volatility neural network for different values of  $\rho$  (left) and  $\sigma$  (right)

We see that the implied volatility smiles produced by the network trained directly on implied volatilities are smoother and more consistent with the FFT benchmark in Figure 7.2, particularly in the deep OTM region where the first network showed irregularities. This shows that the choice of target variable affects the quality of the implied volatility predictions, even when the underlying pricing accuracy is comparable. We also notice that

the model still has a few irregularities, as the network finds it hard to predict implied volatility in the OTM region when  $\sigma$  is high or when  $\rho$  is strongly negative, as both produce faster changes in implied volatility across strikes.

### 7.3 Comparison and Benchmarking

In this section, we compare the neural network and the Carr–Madan FFT method in terms of pricing accuracy at off-grid strikes and computational efficiency.

We begin by comparing the neural network approximation error against the FFT interpolation error at strikes that do not fall on the FFT grid. We use test cases 1, 4 and 5 from Table 7.1 and compute the absolute error of both methods against the explicit Heston price. The results can be seen in Figure 7.7 and Table 7.4.

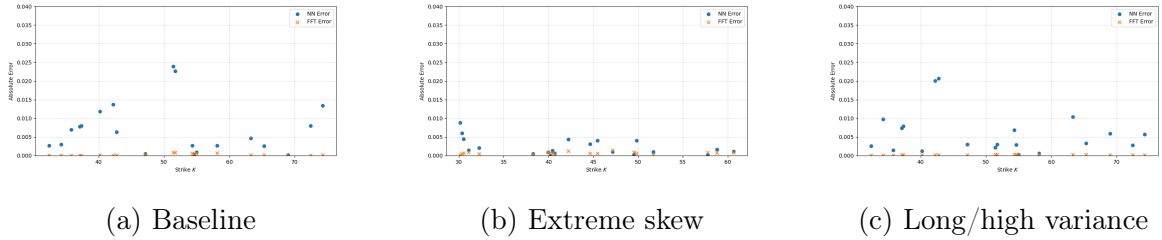


Figure 7.7: Absolute pricing errors of the neural network approximation and FFT interpolation at off the grid strikes against explicit Heston prices

| Method                | Baseline      | Extreme skew  | Long/high variance |
|-----------------------|---------------|---------------|--------------------|
| FFT interpolation MAE | <b>0.0003</b> | <b>0.0007</b> | <b>0.0002</b>      |
| NN approximation MAE  | 0.0072        | 0.0024        | 0.0059             |

Table 7.4: Mean absolute pricing errors of FFT interpolation and neural network approximation at the off grid strikes against explicit Heston prices

We see from the figure that the FFT interpolation error remains close to zero across all strikes in all three cases, while the neural network error is larger but spread relatively evenly across all moneyness regions, which shows that the network does not struggle with any particular region of the strike range. The baseline case shows two outlier points near the ATM region that are noticeably larger than the remaining errors, which can partly be attributed to the structure of the FFT grid used to generate training data, which has relatively few points in the near ATM region. The extreme skew and long/high variance cases show no such concentration, with errors distributed across the full range of strikes. Table 7.4 confirms that FFT interpolation is more accurate, with a MAE that is roughly 3 to 30 times lower than the neural network across all cases.

We now compare the computation time for a single option price. For a fair comparison, all methods in this section are evaluated on CPU (Intel Core i7-1165G7 @ 2.80GHz, 32 GB RAM). The results can be seen in the table below.

| Method                        | Mean (ms)   | Std (ms) |
|-------------------------------|-------------|----------|
| Carr–Madan direct integration | <b>0.29</b> | 0.05     |
| NN single strike              | 0.38        | 0.12     |
| FFT + interpolation           | 1.66        | 0.90     |

Table 7.5: Mean computation time for a single option price across 100 strikes

We see that Carr–Madan direct integration is the fastest method for a single option price at 0.29 ms, which confirms that the neural network does not provide a speed advantage for individual strike evaluations. The FFT is the slowest, as it computes prices for the entire strike grid even when only a single price is needed.

However, the neural network offers a computational advantage when computing an entire volatility surface. For this comparison, we use the FFT rather than direct integration as the benchmark, as direct integration must evaluate a separate integral for each strike, which makes it impractical for computing an entire surface. We generate implied volatility surfaces using the FFT, the pricing neural network, and the implied volatility neural network across 10 maturities and 20 strikes. The surfaces can be seen in Figure 7.8 and the computation times in Table 7.6.

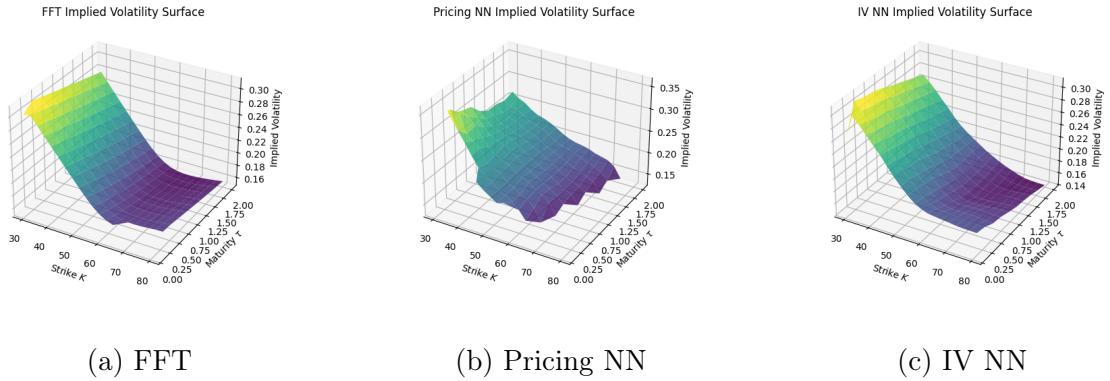


Figure 7.8: Implied volatility surfaces produced by the FFT, pricing neural network, and implied volatility neural network across 10 maturities and 20 strikes

We see that the FFT and IV neural network produce nearly identical surfaces, both showing the expected skew from the negative  $\rho$  and the term structure across maturities. The pricing neural network surface shows visible irregularities, particularly for short maturities and deep OTM strikes, which is consistent with the implied volatility analysis in Section 7.2.3.

| Method              | Mean (ms)   | Std (ms) |
|---------------------|-------------|----------|
| FFT + interpolation | 10.11       | 0.59     |
| Pricing NN          | 3.27        | 0.44     |
| IV NN               | <b>3.26</b> | 0.45     |

Table 7.6: Computation time for a volatility surface (10 maturities  $\times$  20 strikes)

The table above shows that both neural networks are approximately three times as fast as the FFT for computing the volatility surface, with the IV neural network being the fastest at 3.26 ms compared to 10.11 ms for the FFT. This speedup is because the neural network evaluates all 20 strikes in a single pass for each maturity, whereas the FFT computes prices for all  $N = 4096$  grid points per maturity even when only 20 strikes are needed, which means that the FFT also requires interpolation to obtain prices at the desired strikes.

Furthermore, if all 200 inputs are evaluated in a single forward pass rather than one maturity at a time, the neural network computes the surface in 0.26 ms compared to 10.93 ms for the FFT, which is a speedup of approximately 42x. This is because the neural network takes all strike and maturity combinations as a single batch input, whereas the FFT must compute a separate transform for each maturity. This shows that the neural network's advantage increases with the number of simultaneous evaluations. The results can be seen in Appendix [B.1.6](#).

# Chapter 8

## Conclusion

In this chapter, we summarise the key findings of the thesis and answer the research questions presented in Section 1.2. We then discuss the limitations of our approach, followed by directions for future work.

### 8.1 Summary of Findings

The main research question of this thesis was whether a neural network can accurately serve as a surrogate model for the Heston option pricing function when trained on prices generated by the Carr–Madan FFT method, while offering computational advantages over direct FFT evaluation. We address this through three sub-questions.

The first sub-question asked how accurately the neural network can learn the Heston pricing function. We showed in Section 7.2.1 that the network achieves a mean absolute error of 0.0063 for ITM options and 0.0074 for OTM options across five test cases. The mean relative error for ITM and near ATM options is below 0.24%, which shows that the network has learned the pricing function to a high degree of accuracy. The OTM region has a higher mean relative error of 8.49%, which is driven by the low absolute prices rather than large absolute errors. We also showed that training a second neural network directly on implied volatilities produces smoother implied volatility smiles, particularly in the deep OTM region.

The second sub-question asked how the two methods compare in terms of computational efficiency. We showed in Section 7.3 that for a single option price, Carr–Madan direct integration is the fastest method at 0.29 ms, and the neural network does not provide a speed advantage. However, when computing a volatility surface across 10 maturities and 20 strikes, the neural network is approximately three times as fast as the FFT, with the IV neural network computing the surface in 3.26 ms compared to 10.11 ms. This advantage increases to a 42x speedup when all inputs are evaluated in a single batch. We note that the exact speedup depends on the implementation, batching strategy, and hardware, but the main conclusion is that the neural network provides a computational advantage when batch evaluation is used. We also showed that FFT interpolation is roughly 3 to 30 times more accurate than the neural network for off-grid strikes, which means that the neural network trades accuracy for speed. Whether this trade-off is acceptable depends on the application, as real-time risk monitoring may prioritise speed, while final trade pricing may require the higher accuracy of the FFT.



The third sub-question asked whether the neural network can generalize to unseen samples within the Heston parameter space. The network achieves a final validation loss of  $1.28\text{e-}6$  and a mean relative error of  $0.15\%$  on the held-out test set, which shows that it generalizes well to unseen data overall. We showed in Section 7.2.2 through heat maps that the network performs well for maturities  $\tau \in [0.40, 0.75]$  across all values of  $\rho$ , with very short maturities also exhibiting noticeable errors. However, the network shows higher errors for extreme parameter combinations, particularly for high  $\sigma$  combined with long maturities, where the absolute error exceeds 0.5. This indicates that the network generalizes within the training range but struggles where the pricing surface is more complex.

## 8.2 Limitations

The neural network approach has several limitations. The training time is approximately 60 minutes on an NVIDIA Quadro P2000 GPU, which represents a one time cost before the network can be used for pricing.

The network is trained on the Heston model and cannot be applied to other stochastic volatility models without retraining. The parameter space is also limited to the ranges defined in Table 6.2, which means that the network has not seen positive values of  $\rho$  or parameters that violate the Feller condition. Additionally, we only consider European call options, which means that the network cannot price American or put options without modification.

The MSE loss function used during training introduces a bias towards high priced options, as shown in the scatter plot analysis, which reduces the accuracy of the network in the deep OTM region. Alternative loss functions, such as a relative loss or a weighted MSE, could address this but were not explored in this thesis.

## 8.3 Future Work

Several directions could extend the work presented in this thesis. The parameter space could be expanded to include positive values of  $\rho$  and parameters that violate the Feller condition, which would make the network applicable to a wider range of market conditions. Adaptive sampling could be used to generate more training data in regions where the network has high errors, such as long maturities combined with high  $\sigma$ , which could improve accuracy without increasing the total training data significantly.

The approach could also be extended to other stochastic volatility models such as the SABR model, as well as to other option types such as put options and American options. Additionally, rather than replacing the FFT entirely, a neural network could be used to learn the optimal FFT parameters  $\alpha$ ,  $N$ , and  $\eta$  for a given set of Heston parameters, which would allow the two methods to work together rather than as alternatives.

# References

- [1] F. Black and M. Scholes. “The Pricing of Options and Corporate Liabilities.” *Journal of Political Economy*, 81.3, (1973), pp. 637–654.
- [2] S. L. Heston. “A Closed-Form Solution for Options with Stochastic Volatility with Applications to Bond and Currency Options.” *Review of Financial Studies*, 6.2, (1993), pp. 327–343.
- [3] P. Carr and D. B. Madan. “Option valuation using the fast Fourier transform.” *Journal of Computational Finance*, 2.4, (1999), pp. 61–73.
- [4] J. C. Hull. *Options, Futures, and Other Derivatives*. 8th. Pearson, 2013.
- [5] J. C. Cox, J. E. Ingersoll, and S. A. Ross. “A Theory of the Term Structure of Interest Rates.” *Econometrica*, 53.2, (1985), pp. 385–407.
- [6] S. E. Shreve. *Stochastic Calculus for Finance II: Continuous-Time Models*. Springer Finance. New York: Springer, 2004. ISBN: 0-387-40101-6.
- [7] F. D. Rouah. *The Heston Model and Its Extensions in Matlab and C#*. New Jersey: John Wiley & Sons, 2013.
- [8] C. R. Harris et al. “Array programming with NumPy.” *Nature*, 585, (2020), pp. 357–362.
- [9] H. Albrecher, P. Mayer, W. Schoutens, and J. Tistaert. “The Little Heston Trap.” *Wilmott Magazine*, (2007), pp. 83–92.
- [10] I. J. Goodfellow, Y. Bengio, and A. C. Courville. *Deep Learning*. Cambridge, Massachusetts: MIT Press, 2016.
- [11] K. Hornik, M. Stinchcombe, and H. White. “Multilayer Feedforward Networks are Universal Approximators.” *Neural Networks*, 2.5, (1989), pp. 359–366.
- [12] J. M. Hutchinson, A. W. Lo, and T. Poggio. “A Nonparametric Approach to Pricing and Hedging Derivative Securities Via Learning Networks.” *The Journal of Finance*, 49.3, (1994), pp. 851–889.
- [13] S. Liu, C. W. Oosterlee, and S. M. Bohte. “Pricing Options and Computing Implied Volatilities using Neural Networks.” *Risks*, 7.1, (2019), p. 16.
- [14] B. Horvath, A. Muguruza, and M. Tomas. “Deep Learning Volatility: A Deep Neural Network Perspective on Pricing and Calibration in (Rough) Volatility Models.” *Quantitative Finance*, 21.1, (2021), pp. 11–27.
- [15] F. Pedregosa et al. “Scikit-learn: Machine Learning in Python.” *Journal of Machine Learning Research*, 12, (2011), pp. 2825–2830.

- [16] A. Paszke et al. “PyTorch: An Imperative Style, High-Performance Deep Learning Library.” *Advances in Neural Information Processing Systems*, 32, (2019).
- [17] K. He, X. Zhang, S. Ren, and J. Sun. “Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification.” *Proceedings of the IEEE International Conference on Computer Vision*, (2015), pp. 1026–1034.
- [18] L. Prechelt. “Automatic Early Stopping Using Cross Validation: Quantifying the Criteria.” *Neural Networks*, 11.4, (1998), pp. 761–767.
- [19] D. P. Kingma and J. Ba. “Adam: A Method for Stochastic Optimization.” *3rd International Conference on Learning Representations*, (2015).
- [20] A. Choromanska, M. Henaff, M. Mathieu, G. B. Arous, and Y. LeCun. “The Loss Surfaces of Multilayer Networks.” *Proceedings of Machine Learning Research*, 38, (2015), pp. 192–204.

# Appendix A

## Mathematical Results

### A.1 Theorems

**Theorem A.1** (Girsanov, multiple dimensions, [6]). *Let  $T$  be a fixed positive time, and let  $\Theta(t) = (\Theta_1(t), \dots, \Theta_d(t))$  be a  $d$ -dimensional adapted process. Define*

$$Z(t) = \exp \left\{ - \int_0^t \Theta(u) \cdot dW(u) - \frac{1}{2} \int_0^t \|\Theta(u)\|^2 du \right\} \quad (\text{A.1})$$

$$\widetilde{W}(t) = W(t) + \int_0^t \Theta(u) du \quad (\text{A.2})$$

and assume that

$$\mathbb{E} \int_0^T \|\Theta(u)\|^2 Z^2(u) du < \infty$$

Set  $Z = Z(T)$ . Then  $\mathbb{E}Z = 1$ , and under the probability measure  $\widetilde{\mathbb{P}}$  given by

$$\widetilde{\mathbb{P}}(A) = \int_A Z(\omega) d\mathbb{P}(\omega) \quad \text{for all } A \in \mathcal{F},$$

the process  $\widetilde{W}(t)$  is a  $d$ -dimensional Brownian motion.

## A.2 Derivations

### A.2.1 Derivation of the Market Price of Risk

In this derivation, we follow the notation of Theorem A.1 and use  $W$  to denote the Brownian motions, which corresponds to  $Z$  in the main part of the thesis.

Girsanov's theorem tells us that we can shift Brownian motions by adding the old Brownian motion with the drift adjustment  $\Theta(t)$ . This means that we choose  $\Theta(t)$ , so it cancels out the unwanted drift terms.

We start from the real-world dynamics:

$$dS_t = \mu_t S_t dt + \sqrt{v_t} S_t dW_1(t)$$

We now substitute  $dW_1(t) = d\widetilde{W}_1(t) - \Theta_1(t) dt$ , which comes from differentiating equation A.2 and get that

$$dS_t = \mu_t S_t dt + \sqrt{v_t} S_t (d\widetilde{W}_1(t) - \Theta_1(t) dt),$$

and by grouping the terms we get

$$dS_t = (\mu_t - \sqrt{v_t} \Theta_1(t)) S_t dt + \sqrt{v_t} S_t d\widetilde{W}_1(t)$$

Under the risk-neutral condition we have that the drift must be equal to  $(r - q)S_t$ , which gives us  $\mu_t - \sqrt{v_t} \Theta_1(t) = r - q$ . Solving for  $\Theta_1(t)$  we get

$$\Theta_1(t) = \frac{\mu_t - (r - q)}{\sqrt{v_t}}$$

We now do the same for the second SDE

$$dv_t = \kappa(\theta - v_t) dt + \sigma\sqrt{v_t} dW_2(t)$$

We substitute  $dW_2(t) = d\widetilde{W}_2(t) - \Theta_2(t) dt$ , which gives us

$$dv_t = \kappa(\theta - v_t) dt + \sigma\sqrt{v_t} (d\widetilde{W}_2(t) - \Theta_2(t) dt)$$

The drift becomes  $\kappa(\theta - v_t) - \sigma\sqrt{v_t} \Theta_2(t)$ . Unlike the stock price dynamics, the variance process is not directly traded, so no-arbitrage does not uniquely determine  $\Theta_2(t)$ .

Following Heston [2], we assume that the volatility risk premium takes the form  $\lambda(S_t, v_t, t) = \lambda v_t$ , where  $\lambda$  is a constant. Since  $\lambda(S_t, v_t, t) \equiv \sigma\sqrt{v_t} \Theta_2(t)$ , this implies that

$$\Theta_2(t) = \frac{\lambda\sqrt{v_t}}{\sigma}$$

# Appendix B

## Computational Results

### B.1 Tables

#### B.1.1 Numerical Stability Results of FFT

In this appendix the results of the FFT algorithm are reported. The absolute error is computed against the benchmark price, which is calculated using Gil-Pelaez inversion formula.

| Option               | $S$ | $K$ | $\tau$ | $\kappa$ | $\theta$ | $\sigma$ | $\rho$ | $v_0$ | $r$  | $q$  | Trap |
|----------------------|-----|-----|--------|----------|----------|----------|--------|-------|------|------|------|
| ATM option           | 100 | 100 | 0.5    | 5        | 0.05     | 0.5      | -0.8   | 0.05  | 0.03 | 0.02 | 1    |
| ITM option           | 100 | 80  | 0.5    | 5        | 0.05     | 0.5      | -0.8   | 0.05  | 0.03 | 0.02 | 1    |
| OTM option           | 100 | 120 | 0.5    | 5        | 0.05     | 0.5      | -0.8   | 0.05  | 0.03 | 0.02 | 1    |
| Long maturity option | 100 | 100 | 2.0    | 5        | 0.05     | 0.5      | -0.8   | 0.05  | 0.03 | 0.02 | 1    |
| High vol of vol      | 100 | 100 | 2.0    | 5        | 0.05     | 1.0      | -0.8   | 0.05  | 0.03 | 0.02 | 1    |

Table B.1: Input parameters for test cases

#### ATM Option

Benchmark price: 6.252819

##### Alpha sensitivity

| $\alpha$ | Error    |
|----------|----------|
| 0.50     | 6.14e-02 |
| 1.00     | 2.56e-04 |
| 1.25     | 1.46e-04 |
| 1.50     | 1.41e-04 |
| 1.75     | 1.41e-04 |
| 2.00     | 1.41e-04 |
| 2.50     | 1.41e-04 |
| 3.00     | 1.41e-04 |

##### Grid size sensitivity

| $N$  | Error    | Time (s) |
|------|----------|----------|
| 256  | 1.41e-04 | 0.0002   |
| 512  | 1.41e-04 | 0.0002   |
| 1024 | 1.41e-04 | 0.0003   |
| 2048 | 1.41e-04 | 0.0005   |
| 4096 | 1.41e-04 | 0.0011   |
| 8192 | 1.41e-04 | 0.0022   |

##### Damping parameter sensitivity

| $\eta$ | Error    |
|--------|----------|
| 0.01   | 1.52e-04 |
| 0.05   | 1.41e-04 |
| 0.10   | 1.41e-04 |
| 0.25   | 1.41e-04 |
| 0.50   | 2.80e-03 |
| 1.00   | 2.76e-01 |

### ITM Option

Benchmark price: 21.047231

#### Alpha sensitivity

| $\alpha$ | Error    |
|----------|----------|
| 0.50     | 6.09e-02 |
| 1.00     | 2.25e-04 |
| 1.25     | 3.35e-04 |
| 1.50     | 3.40e-04 |
| 1.75     | 3.40e-04 |
| 2.00     | 3.40e-04 |
| 2.50     | 3.40e-04 |
| 3.00     | 3.40e-04 |

#### Grid size sensitivity

| $N$  | Error    | Time (s) |
|------|----------|----------|
| 256  | 5.21e-02 | 0.0001   |
| 512  | 1.83e-02 | 0.0002   |
| 1024 | 1.52e-03 | 0.0003   |
| 2048 | 7.44e-04 | 0.0005   |
| 4096 | 3.40e-04 | 0.0012   |
| 8192 | 1.33e-04 | 0.0022   |

#### Damping parameter sensitivity

| $\eta$ | Error    |
|--------|----------|
| 0.01   | 1.74e-01 |
| 0.05   | 5.52e-03 |
| 0.10   | 1.84e-03 |
| 0.25   | 3.40e-04 |
| 0.50   | 2.53e-03 |
| 1.00   | 2.78e-01 |

### OTM Option

Benchmark price: 0.451158

#### Alpha sensitivity

| $\alpha$ | Error    |
|----------|----------|
| 0.50     | 6.04e-02 |
| 1.00     | 8.12e-04 |
| 1.25     | 9.22e-04 |
| 1.50     | 9.27e-04 |
| 1.75     | 9.27e-04 |
| 2.00     | 9.27e-04 |
| 2.50     | 9.27e-04 |
| 3.00     | 9.27e-04 |

#### Grid size sensitivity

| $N$  | Error    | Time (s) |
|------|----------|----------|
| 256  | 1.36e-01 | 0.0003   |
| 512  | 4.99e-02 | 0.0002   |
| 1024 | 1.39e-02 | 0.0004   |
| 2048 | 1.99e-03 | 0.0006   |
| 4096 | 9.27e-04 | 0.0012   |
| 8192 | 4.12e-04 | 0.0023   |

#### Damping parameter sensitivity

| $\eta$ | Error    |
|--------|----------|
| 0.01   | 2.33e-01 |
| 0.05   | 5.43e-03 |
| 0.10   | 2.54e-03 |
| 0.25   | 9.27e-04 |
| 0.50   | 2.24e-03 |
| 1.00   | 2.73e-01 |

## Long Maturity Option

Benchmark price: 12.620743

### Alpha sensitivity

| $\alpha$ | Error    |
|----------|----------|
| 0.50     | 5.94e-02 |
| 1.00     | 8.23e-05 |
| 1.25     | 2.45e-05 |
| 1.50     | 2.92e-05 |
| 1.75     | 2.94e-05 |
| 2.00     | 2.94e-05 |
| 2.50     | 2.94e-05 |
| 3.00     | 2.94e-05 |

### Grid size sensitivity

| $N$  | Error    | Time (s) |
|------|----------|----------|
| 256  | 2.92e-05 | 0.0005   |
| 512  | 2.92e-05 | 0.0003   |
| 1024 | 2.92e-05 | 0.0004   |
| 2048 | 2.92e-05 | 0.0006   |
| 4096 | 2.92e-05 | 0.0010   |
| 8192 | 2.92e-05 | 0.0029   |

### Damping parameter sensitivity

| $\eta$ | Error    |
|--------|----------|
| 0.01   | 2.94e-05 |
| 0.05   | 2.94e-05 |
| 0.10   | 2.94e-05 |
| 0.25   | 2.92e-05 |
| 0.50   | 2.55e-03 |
| 1.00   | 2.68e-01 |

## High Vol of Vol

Benchmark price: 12.073467

### Alpha sensitivity

| $\alpha$ | Error    |
|----------|----------|
| 0.50     | 5.94e-02 |
| 1.00     | 8.28e-05 |
| 1.25     | 2.41e-05 |
| 1.50     | 2.87e-05 |
| 1.75     | 2.89e-05 |
| 2.00     | 2.89e-05 |
| 2.50     | 2.89e-05 |
| 3.00     | 2.89e-05 |

### Grid size sensitivity

| $N$  | Error    | Time (s) |
|------|----------|----------|
| 256  | 2.87e-05 | 0.0004   |
| 512  | 2.87e-05 | 0.0002   |
| 1024 | 2.87e-05 | 0.0003   |
| 2048 | 2.87e-05 | 0.0006   |
| 4096 | 2.87e-05 | 0.0013   |
| 8192 | 2.87e-05 | 0.0027   |

### Damping parameter sensitivity

| $\eta$ | Error    |
|--------|----------|
| 0.01   | 2.82e-05 |
| 0.05   | 2.89e-05 |
| 0.10   | 2.89e-05 |
| 0.25   | 2.87e-05 |
| 0.50   | 2.55e-03 |
| 1.00   | 2.68e-01 |



### B.1.2 Architecture Logs

In this appendix the full log results of the architecture selection can be seen for seed 5. The remaining seeds (40 and 120) show similar convergence behavior and are available upon request.

| Epoch | Train. Loss | Val. Loss | LR      |
|-------|-------------|-----------|---------|
| 10    | 2.689e-5    | 2.645e-5  | 1e-3    |
| 20    | 1.404e-5    | 3.495e-5  | 1e-3    |
| 30    | 1.033e-5    | 6.115e-6  | 1e-3    |
| 40    | 8.311e-6    | 3.467e-6  | 1e-3    |
| 50    | 7.327e-6    | 5.533e-6  | 1e-3    |
| 60    | 1.860e-6    | 4.241e-6  | 5e-4    |
| 70    | 2.400e-6    | 2.830e-6  | 5e-4    |
| 80    | 2.262e-6    | 2.207e-6  | 5e-4    |
| 90    | 2.175e-6    | 2.753e-6  | 5e-4    |
| 100   | 1.158e-6    | 2.002e-6  | 2.5e-4  |
| 110   | 1.107e-6    | 2.286e-6  | 2.5e-4  |
| 120   | 1.073e-6    | 1.711e-6  | 1.25e-4 |
| 130   | 7.680e-7    | 1.599e-6  | 1.25e-4 |
| 140   | 7.611e-7    | 1.503e-6  | 1.25e-4 |
| 150   | 7.432e-7    | 1.635e-6  | 1.25e-4 |
| 160   | 6.372e-7    | 1.526e-6  | 6.25e-5 |
| 170   | 6.275e-7    | 1.574e-6  | 6.25e-5 |
| 180   | 5.806e-7    | 1.418e-6  | 3.13e-5 |
| 190   | 5.763e-7    | 1.406e-6  | 3.13e-5 |
| 200   | 5.519e-7    | 1.382e-6  | 1.56e-5 |

Table B.2: Training log for  $3 \times 128$

| Epoch | Train. Loss | Val. Loss | LR      |
|-------|-------------|-----------|---------|
| 10    | 3.689e-5    | 2.506e-5  | 1e-3    |
| 20    | 2.189e-5    | 1.556e-5  | 1e-3    |
| 30    | 1.611e-5    | 9.943e-6  | 1e-3    |
| 40    | 1.335e-5    | 1.024e-5  | 1e-3    |
| 50    | 4.964e-6    | 5.655e-6  | 5e-4    |
| 60    | 5.339e-6    | 8.333e-6  | 5e-4    |
| 70    | 4.947e-6    | 8.571e-6  | 5e-4    |
| 80    | 4.640e-6    | 5.142e-6  | 5e-4    |
| 90    | 4.451e-6    | 7.443e-6  | 5e-4    |
| 100   | 2.837e-6    | 3.270e-6  | 2.5e-4  |
| 110   | 2.760e-6    | 5.037e-6  | 2.5e-4  |
| 120   | 2.642e-6    | 3.090e-6  | 2.5e-4  |
| 130   | 2.602e-6    | 3.246e-6  | 2.5e-4  |
| 140   | 2.102e-6    | 2.806e-6  | 1.25e-4 |
| 150   | 2.057e-6    | 2.816e-6  | 1.25e-4 |
| 160   | 2.037e-6    | 2.961e-6  | 1.25e-4 |
| 170   | 1.829e-6    | 2.457e-6  | 6.25e-5 |
| 180   | 1.812e-6    | 2.404e-6  | 6.25e-5 |
| 190   | 1.788e-6    | 2.478e-6  | 6.25e-5 |
| 200   | 1.770e-6    | 2.563e-6  | 6.25e-5 |

---

| Epoch | Train. Loss | Val. Loss | LR      |
|-------|-------------|-----------|---------|
| 10    | 3.300e-5    | 6.853e-5  | 1e-3    |
| 20    | 1.661e-5    | 6.947e-6  | 1e-3    |
| 30    | 1.155e-5    | 1.308e-5  | 1e-3    |
| 40    | 2.401e-6    | 1.111e-5  | 5e-4    |
| 50    | 3.179e-6    | 3.958e-6  | 5e-4    |
| 60    | 1.505e-6    | 2.192e-6  | 2.5e-4  |
| 70    | 1.387e-6    | 1.582e-6  | 2.5e-4  |
| 80    | 1.353e-6    | 2.094e-6  | 2.5e-4  |
| 90    | 1.282e-6    | 1.711e-6  | 1.25e-4 |
| 100   | 8.333e-7    | 1.391e-6  | 1.25e-4 |
| 110   | 8.022e-7    | 1.512e-6  | 1.25e-4 |
| 120   | 6.552e-7    | 1.333e-6  | 6.25e-5 |
| 130   | 6.407e-7    | 1.290e-6  | 6.25e-5 |
| 140   | 6.243e-7    | 1.248e-6  | 6.25e-5 |
| 150   | 6.139e-7    | 1.205e-6  | 6.25e-5 |
| 160   | 5.998e-7    | 1.242e-6  | 6.25e-5 |
| 170   | 5.441e-7    | 1.214e-6  | 3.13e-5 |
| 180   | 5.382e-7    | 1.183e-6  | 3.13e-5 |
| 190   | 5.086e-7    | 1.163e-6  | 1.56e-5 |
| 200   | 5.057e-7    | 1.169e-6  | 7.81e-6 |

Table B.3: Training logs for  $4 \times 64$  (left) and  $4 \times 128$  (right)

| Epoch | Train. Loss | Val. Loss | LR      |
|-------|-------------|-----------|---------|
| 10    | 3.561e-5    | 1.041e-5  | 1e-3    |
| 20    | 1.703e-5    | 1.188e-5  | 1e-3    |
| 30    | 3.996e-6    | 2.780e-6  | 5e-4    |
| 40    | 3.516e-6    | 4.218e-6  | 5e-4    |
| 50    | 3.214e-6    | 4.398e-6  | 5e-4    |
| 60    | 2.909e-6    | 6.308e-6  | 2.5e-4  |
| 70    | 1.141e-6    | 1.298e-6  | 2.5e-4  |
| 80    | 1.092e-6    | 1.272e-6  | 2.5e-4  |
| 90    | 1.057e-6    | 3.051e-6  | 2.5e-4  |
| 100   | 1.002e-6    | 1.305e-6  | 2.5e-4  |
| 110   | 5.551e-7    | 1.094e-6  | 1.25e-4 |
| 120   | 5.329e-7    | 1.303e-6  | 1.25e-4 |
| 130   | 5.289e-7    | 1.069e-6  | 1.25e-4 |
| 140   | 4.938e-7    | 1.185e-6  | 1.25e-4 |
| 150   | 3.715e-7    | 1.036e-6  | 6.25e-5 |
| 160   | 3.623e-7    | 1.102e-6  | 6.25e-5 |
| 170   | 3.576e-7    | 9.904e-7  | 6.25e-5 |
| 180   | 3.090e-7    | 9.560e-7  | 3.13e-5 |
| 190   | 3.060e-7    | 9.487e-7  | 3.13e-5 |
| 200   | 3.014e-7    | 1.057e-6  | 3.13e-5 |

---

| Epoch | Train. Loss | Val. Loss | LR      |
|-------|-------------|-----------|---------|
| 10    | 3.663e-5    | 1.258e-5  | 1e-3    |
| 20    | 1.993e-5    | 2.033e-5  | 1e-3    |
| 30    | 1.517e-5    | 5.330e-6  | 1e-3    |
| 40    | 1.110e-5    | 1.134e-5  | 1e-3    |
| 50    | 9.265e-6    | 3.474e-6  | 1e-3    |
| 60    | 9.078e-6    | 4.641e-6  | 1e-3    |
| 70    | 7.487e-6    | 2.650e-5  | 1e-3    |
| 80    | 2.607e-6    | 2.499e-6  | 5e-4    |
| 90    | 2.450e-6    | 2.212e-6  | 5e-4    |
| 100   | 1.204e-6    | 1.617e-6  | 2.5e-4  |
| 110   | 1.153e-6    | 1.574e-6  | 2.5e-4  |
| 120   | 7.289e-7    | 1.462e-6  | 1.25e-4 |
| 130   | 7.604e-7    | 1.373e-6  | 1.25e-4 |
| 140   | 7.369e-7    | 1.373e-6  | 6.25e-5 |
| 150   | 6.199e-7    | 1.389e-6  | 6.25e-5 |
| 160   | 6.082e-7    | 1.315e-6  | 6.25e-5 |
| 170   | 5.548e-7    | 1.272e-6  | 3.13e-5 |
| 180   | 5.472e-7    | 1.290e-6  | 3.13e-5 |
| 190   | 5.424e-7    | 1.276e-6  | 3.13e-5 |
| 200   | 5.361e-7    | 1.260e-6  | 3.13e-5 |

Table B.4: Training logs for  $4 \times 256$  (left) and  $5 \times 128$  (right)

### B.1.3 Training Logs

In this appendix the full log results of the training can be seen for the batch sizes, 1024, 2048, 4096, 8192 and 16384.

| Epoch | Train. Loss | Val. Loss | LR    | Epoch | Train. Loss | Val. Loss | LR    |
|-------|-------------|-----------|-------|-------|-------------|-----------|-------|
| 0     | 7.515e-3    | 3.634e-4  | 0.001 | 0     | 1.388e-2    | 5.846e-4  | 0.001 |
| 10    | 2.867e-5    | 8.432e-5  | 0.001 | 10    | 5.647e-5    | 3.980e-5  | 0.001 |
| 20    | 1.606e-5    | 2.002e-5  | 0.001 | 20    | 2.578e-5    | 1.931e-5  | 0.001 |
| 30    | 1.164e-5    | 1.055e-5  | 0.001 | 30    | 1.877e-5    | 6.176e-6  | 0.001 |
| 40    | 1.624e-6    | 2.280e-6  | 1e-4  | 40    | 1.234e-5    | 1.001e-5  | 0.001 |
| 50    | 1.323e-6    | 2.415e-6  | 1e-4  | 50    | 1.252e-5    | 4.065e-6  | 0.001 |
| 60    | 1.180e-6    | 1.765e-6  | 1e-4  | 60    | 1.122e-5    | 3.777e-6  | 0.001 |
| 70    | 1.080e-6    | 1.842e-6  | 1e-4  | 70    | 1.024e-5    | 1.355e-5  | 0.001 |
| 80    | 1.004e-6    | 1.759e-6  | 1e-4  | 80    | 1.375e-6    | 2.223e-6  | 1e-4  |
| 90    | 9.381e-7    | 1.640e-6  | 1e-4  | 90    | 1.216e-6    | 2.174e-6  | 1e-4  |
| 100   | 8.929e-7    | 1.562e-6  | 1e-4  | 100   | 1.122e-6    | 1.994e-6  | 1e-4  |
| 110   | 8.594e-7    | 1.503e-6  | 1e-4  | 110   | 1.049e-6    | 1.870e-6  | 1e-4  |
| 120   | 8.186e-7    | 1.621e-6  | 1e-4  | 120   | 9.950e-7    | 1.825e-6  | 1e-4  |
| 130   | 7.994e-7    | 1.401e-6  | 1e-4  | 130   | 9.583e-7    | 1.871e-6  | 1e-4  |
| 140   | 6.119e-7    | 1.307e-6  | 1e-5  | 140   | 9.156e-7    | 1.762e-6  | 1e-4  |
| 150   | 6.054e-7    | 1.310e-6  | 1e-5  | 150   | 8.677e-7    | 1.647e-6  | 1e-4  |
| 160   | 5.845e-7    | 1.291e-6  | 1e-6  | 160   | 8.446e-7    | 1.738e-6  | 1e-4  |
| 170   | 5.840e-7    | 1.292e-6  | 1e-6  | 170   | 6.968e-7    | 1.549e-6  | 1e-5  |
| 180   | 5.834e-7    | 1.288e-6  | 1e-6  | 180   | 6.906e-7    | 1.541e-6  | 1e-5  |
| 190   | 5.828e-7    | 1.285e-6  | 1e-6  | 190   | 6.865e-7    | 1.524e-6  | 1e-5  |

Table B.5: Training logs for batch sizes 1024 (left) and 2048 (right)

| Epoch | Train. Loss | Val. Loss | LR    | Epoch | Train. Loss | Val. Loss | LR    |
|-------|-------------|-----------|-------|-------|-------------|-----------|-------|
| 0     | 2.639e-2    | 1.225e-3  | 0.001 | 0     | 4.893e-2    | 2.445e-3  | 0.001 |
| 10    | 1.066e-4    | 5.368e-5  | 0.001 | 10    | 2.437e-4    | 1.759e-4  | 0.001 |
| 20    | 4.965e-5    | 1.140e-4  | 0.001 | 20    | 1.151e-4    | 5.438e-4  | 0.001 |
| 30    | 3.378e-5    | 7.914e-5  | 0.001 | 30    | 7.800e-5    | 4.306e-5  | 0.001 |
| 40    | 2.702e-5    | 9.887e-6  | 0.001 | 40    | 5.226e-5    | 3.859e-5  | 0.001 |
| 50    | 1.936e-5    | 1.511e-5  | 0.001 | 50    | 4.864e-5    | 7.001e-5  | 0.001 |
| 60    | 1.753e-5    | 3.295e-5  | 0.001 | 60    | 3.715e-5    | 7.605e-5  | 0.001 |
| 70    | 1.400e-5    | 5.719e-6  | 0.001 | 70    | 2.378e-5    | 1.416e-5  | 0.001 |
| 80    | 1.618e-5    | 3.999e-6  | 0.001 | 80    | 2.381e-5    | 2.296e-5  | 0.001 |
| 90    | 1.330e-5    | 1.888e-5  | 0.001 | 90    | 2.065e-5    | 1.555e-5  | 0.001 |
| 100   | 1.940e-6    | 2.962e-6  | 1e-4  | 100   | 1.573e-5    | 1.459e-5  | 0.001 |
| 110   | 1.782e-6    | 2.642e-6  | 1e-4  | 110   | 1.502e-5    | 6.832e-6  | 0.001 |
| 120   | 1.644e-6    | 2.758e-6  | 1e-4  | 120   | 1.893e-5    | 2.965e-5  | 0.001 |
| 130   | 1.532e-6    | 2.448e-6  | 1e-4  | 130   | 1.615e-5    | 2.796e-5  | 0.001 |
| 140   | 1.444e-6    | 2.645e-6  | 1e-4  | 140   | 1.410e-5    | 5.495e-5  | 1e-4  |
| 150   | 1.388e-6    | 2.687e-6  | 1e-4  | 150   | 2.571e-6    | 3.672e-6  | 1e-4  |
| 160   | 1.315e-6    | 2.264e-6  | 1e-4  | 160   | 2.515e-6    | 3.430e-6  | 1e-4  |
| 170   | 1.268e-6    | 2.054e-6  | 1e-4  | 170   | 2.275e-6    | 3.227e-6  | 1e-4  |
| 180   | 1.082e-6    | 1.958e-6  | 1e-5  | 180   | 2.164e-6    | 3.096e-6  | 1e-4  |
| 190   | 1.079e-6    | 1.953e-6  | 1e-5  | 190   | 2.054e-6    | 3.000e-6  | 1e-4  |

Table B.6: Training logs for batch sizes 4096 (left) and 8192 (right)

| Epoch | Train. Loss | Val. Loss | LR    |
|-------|-------------|-----------|-------|
| 0     | 1.011e-1    | 4.529e-3  | 0.001 |
| 10    | 5.643e-4    | 4.523e-4  | 0.001 |
| 20    | 2.331e-4    | 1.793e-4  | 0.001 |
| 30    | 1.723e-4    | 1.043e-4  | 0.001 |
| 40    | 1.432e-4    | 6.572e-5  | 0.001 |
| 50    | 8.370e-5    | 5.272e-5  | 0.001 |
| 60    | 9.758e-5    | 3.823e-5  | 0.001 |
| 70    | 7.939e-5    | 3.151e-5  | 0.001 |
| 80    | 6.714e-5    | 2.995e-4  | 0.001 |
| 90    | 4.820e-5    | 1.944e-5  | 0.001 |
| 100   | 3.533e-5    | 8.602e-5  | 0.001 |
| 110   | 5.654e-5    | 1.061e-4  | 0.001 |
| 120   | 2.094e-5    | 1.795e-5  | 0.001 |
| 130   | 7.742e-6    | 9.676e-6  | 1e-4  |
| 140   | 7.278e-6    | 9.056e-6  | 1e-4  |
| 150   | 6.733e-6    | 8.461e-6  | 1e-4  |
| 160   | 6.425e-6    | 7.681e-6  | 1e-4  |
| 170   | 5.866e-6    | 8.355e-6  | 1e-4  |
| 180   | 5.414e-6    | 6.585e-6  | 1e-4  |
| 190   | 5.243e-6    | 6.174e-6  | 1e-4  |

Table B.7: Training log for batch size 16384

### B.1.4 Results for Implied Volatility Neural Network

This appendix presents the training log and performance metrics for the implied volatility neural network described in Section 7.2.3.

| Epoch | Train. Loss | Val. Loss | LR       |
|-------|-------------|-----------|----------|
| 0     | 1.239e-2    | 1.521e-3  | 0.001000 |
| 10    | 4.986e-4    | 4.807e-4  | 0.001000 |
| 20    | 3.743e-4    | 3.682e-4  | 0.001000 |
| 30    | 3.226e-4    | 2.940e-4  | 0.001000 |
| 40    | 2.916e-4    | 2.667e-4  | 0.001000 |
| 50    | 2.631e-4    | 2.987e-4  | 0.001000 |
| 60    | 2.556e-4    | 4.584e-4  | 0.001000 |
| 70    | 2.367e-4    | 2.529e-4  | 0.001000 |
| 80    | 2.284e-4    | 3.466e-4  | 0.001000 |
| 90    | 2.196e-4    | 2.453e-4  | 0.001000 |
| 100   | 2.175e-4    | 2.376e-4  | 0.001000 |
| 110   | 2.047e-4    | 2.286e-4  | 0.001000 |
| 120   | 1.389e-4    | 1.982e-4  | 0.000500 |
| 130   | 1.393e-4    | 2.317e-4  | 0.000500 |
| 140   | 1.329e-4    | 1.598e-4  | 0.000500 |
| 150   | 1.349e-4    | 1.691e-4  | 0.000500 |
| 160   | 1.284e-4    | 1.594e-4  | 0.000500 |
| 170   | 9.121e-5    | 1.343e-4  | 0.000250 |
| 180   | 9.184e-5    | 1.275e-4  | 0.000250 |
| 190   | 8.854e-5    | 1.166e-4  | 0.000250 |

Table B.8: Training log for the implied volatility neural network with batch size 1024

| Metric                    | 1024     |
|---------------------------|----------|
| Mean relative error (%)   | 0.13     |
| Median relative error (%) | 0.08     |
| Max relative error (%)    | 20.81    |
| Samples evaluated         | 226459   |
| Mean absolute error       | 0.0356   |
| Median absolute error     | 0.0251   |
| Max absolute error        | 5.0067   |
| Final training loss       | 8.854e-5 |
| Final validation loss     | 1.166e-4 |

Table B.9: Performance of the implied volatility neural network with batch size 1024

### B.1.5 Neural network call option prices and Carr–Madan FFT benchmark price error metrics

In this appendix the detailed error metrics for the test cases in Table 7.1 can be seen. The near ATM region is defined as strikes within 5% of the spot price, which gives 16 strikes per test case. The OTM regions contain 44-69 strikes, depending on the test case and ITM region contains 75 strikes.

| Metric    | OTM    | Near ATM | ITM    |
|-----------|--------|----------|--------|
| MAE       | 0.0075 | 0.0210   | 0.0076 |
| Median AE | 0.0046 | 0.0233   | 0.0075 |
| RMSE      | 0.0108 | 0.0221   | 0.0085 |
| MRE (%)   | 12.07  | 0.74     | 0.08   |
| Max AE    | 0.0299 | 0.0292   | 0.0161 |

Table B.10: Neural network pricing errors for the baseline benchmark case, grouped by moneyness

| Metric    | OTM    | Near ATM | ITM    |
|-----------|--------|----------|--------|
| MAE       | 0.0081 | 0.0033   | 0.0025 |
| Median AE | 0.0031 | 0.0027   | 0.0017 |
| RMSE      | 0.0131 | 0.0039   | 0.0034 |
| MRE (%)   | 14.37  | 0.14     | 0.03   |
| Max AE    | 0.0369 | 0.0083   | 0.0084 |

Table B.11: Neural network pricing errors for the short-maturity high-variance case, grouped by moneyness

| Metric    | OTM    | Near ATM | ITM    |
|-----------|--------|----------|--------|
| MAE       | 0.0091 | 0.0071   | 0.0101 |
| Median AE | 0.0049 | 0.0052   | 0.0065 |
| RMSE      | 0.0128 | 0.0096   | 0.0134 |
| MRE (%)   | 2.07   | 0.17     | 0.11   |
| Max AE    | 0.0291 | 0.0196   | 0.0323 |

Table B.12: Neural network pricing errors for the long-maturity low-vol-of-vol case, grouped by moneyness

| Metric    | OTM    | Near ATM | ITM    |
|-----------|--------|----------|--------|
| MAE       | 0.0075 | 0.0020   | 0.0023 |
| Median AE | 0.0030 | 0.0017   | 0.0017 |
| RMSE      | 0.0114 | 0.0024   | 0.0030 |
| MRE (%)   | 13.73  | 0.12     | 0.02   |
| Max AE    | 0.0298 | 0.0052   | 0.0107 |

Table B.13: Neural network pricing errors for the extreme short-maturity skew case, grouped by moneyness

| Metric    | OTM    | Near ATM | ITM    |
|-----------|--------|----------|--------|
| MAE       | 0.0046 | 0.0036   | 0.0091 |
| Median AE | 0.0039 | 0.0035   | 0.0081 |
| RMSE      | 0.0058 | 0.0044   | 0.0114 |
| MRE (%)   | 0.22   | 0.05     | 0.08   |
| Max AE    | 0.0162 | 0.0099   | 0.0231 |

Table B.14: Neural network pricing errors for the long-maturity high-variance case, grouped by moneyness

### B.1.6 Batch Computation Time

In this appendix we report the computation time for generating a volatility surface when all 200 inputs (10 maturities  $\times$  20 strikes) are evaluated in a single forward pass for the neural network.

| Method              | Mean (ms) | Std (ms) |
|---------------------|-----------|----------|
| FFT + interpolation | 10.9274   | 1.2919   |
| NN single batch     | 0.2602    | 0.0475   |

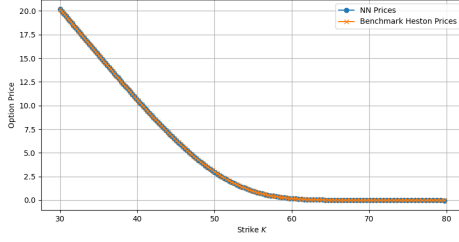
Table B.15: Computation time for a volatility surface with all inputs evaluated in a single batch



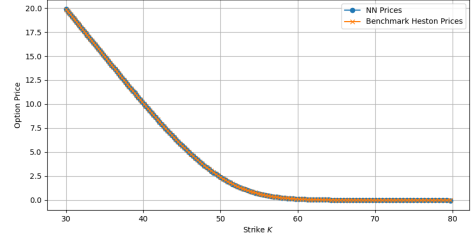
## B.2 Figures

### B.2.1 Neural network call option prices and Carr–Madan FFT benchmark prices

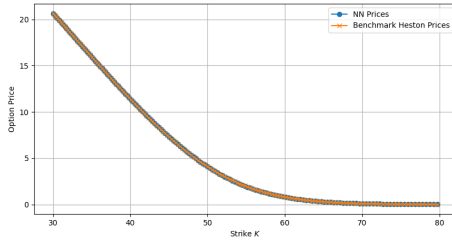
In this appendix the curve for the test cases in Table 7.1 can be seen, where it is represented as a function of the strike  $K$ . In this appendix we also show the plots on a logarithmic scale to capture the behavior of option prices in deep OTM region more clearly.



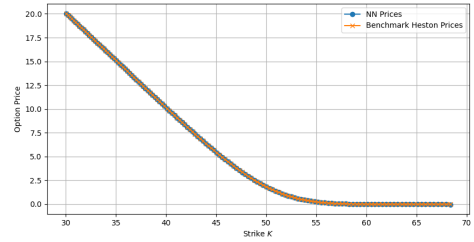
(a) Baseline benchmark



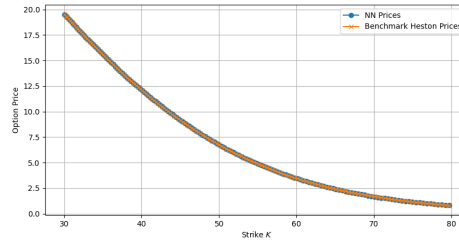
(b) Short maturity/high variance



(c) Long maturity/low vol-of-vol

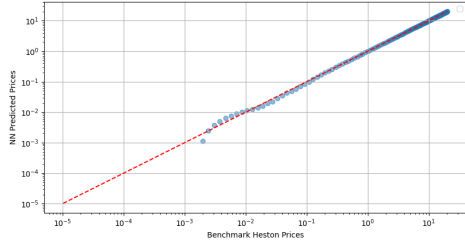


(d) Extreme short-maturity skew

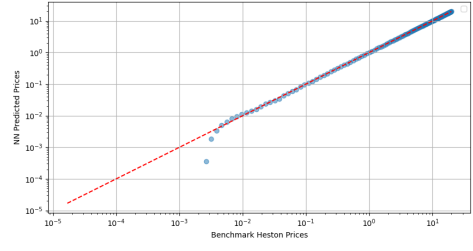


(e) Long maturity/high variance

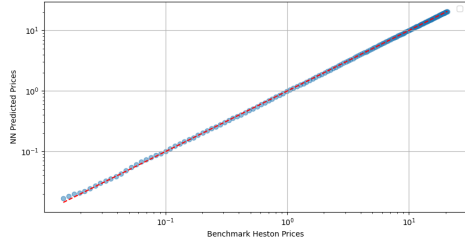
Figure B.1: Neural network call option prices and Carr–Madan FFT benchmark prices across five parameter cases



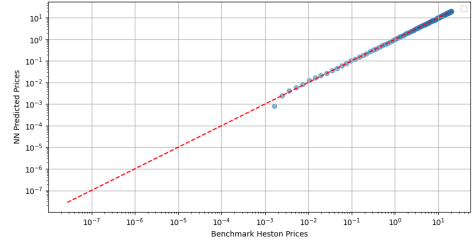
(a) Baseline



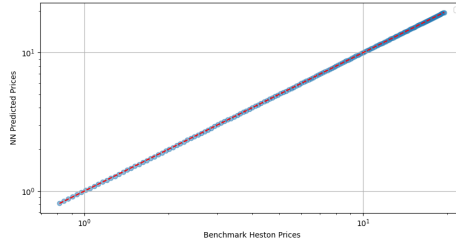
(b) Short/high variance



(c) Long/low vol-of-vol

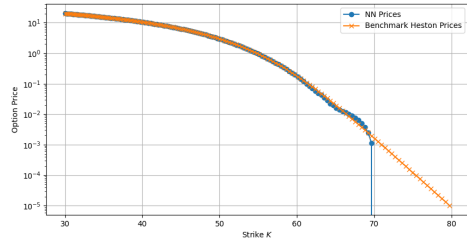


(d) Extreme skew

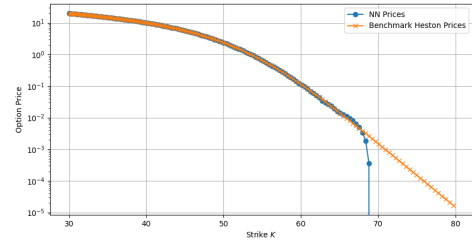


(e) Long/high variance

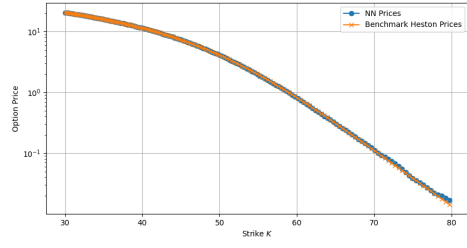
Figure B.2: Neural network predicted prices against Carr–Madan FFT benchmark prices across five representative Heston parameter cases on logarithmic scale



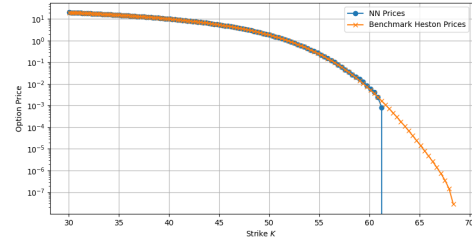
(a) Baseline benchmark



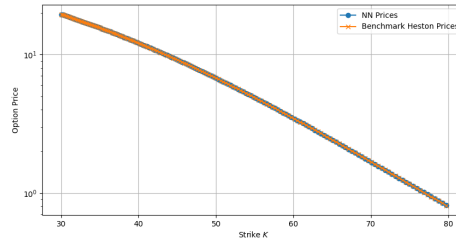
(b) Short maturity/high variance



(c) Long maturity/low vol-of-vol



(d) Extreme short-maturity skew



(e) Long maturity/high variance

Figure B.3: Neural network call option prices and Carr–Madan FFT benchmark prices across five parameter cases on logarithmic scale

# Appendix C

## Source Code

The full source code is available at <https://github.com/manan071/heston-ml-bachelor>.

### C.1 Carr–Madan FFT Implementation

```
1 import numpy as np
2
3 i = 1j
4
5 def heston_characteristic_function_cm(u, S, tau, kappa, theta,
6   sigma, rho, v0, r, q, trap):
7     x = np.log(S)
8     a = kappa*theta
9     u_heston = -0.5
10    b = kappa
11
12    d = np.sqrt((rho*sigma*i*u-b)**2-sigma**2*(2*u_heston*i*u-u**2)
13              )
14    g = (b-rho*sigma*i*u+d)/(b-rho*sigma*i*u-d)
15
16    if trap == 1:
17        c = 1/g
18        D = (b-rho*sigma*i*u-d)/sigma**2*((1-np.exp(-d*tau))/(1-c*
19          np.exp(-d*tau)))
20        G = (1-c*np.exp(-d*tau))/(1-c)
21        C = (r-q)*i*u*tau+a/sigma**2*((b-rho*sigma*i*u-d)*tau-2*np.
22          log(G))
23    else:
24        G = (1-g*np.exp(d*tau))/(1-g)
25        C = (r-q)*i*u*tau+a/sigma**2*((b-rho*sigma*i*u+d)*tau-2*np.
26          log(G))
27        D = (b-rho*sigma*i*u+d)/sigma**2*((1-np.exp(d*tau))/(1-g*np.
28          exp(d*tau)))
29
30    return np.exp(C+D*v0+i*u*x)
```

```

26 def heston_call_FFT(N, eta, alpha, S, tau, kappa, theta, sigma, rho
    , v0, r, q, trap):
27     j = np.arange(N)
28     s0 = np.log(S)
29     v = np.arange(N)*eta
30     lambdainc = 2*np.pi/(N*eta)
31     b = N*lambdainc/2
32     k = -b+j*lambdainc+s0
33
34     w = np.ones(N)
35     w[0] = 1/3
36     w[-1] = 1/3
37     w[1:-1:2] = 4/3
38     w[2:-1:2] = 2/3
39
40     f2 = heston_characteristic_function_cm(v-(alpha+1)*i, S, tau,
        kappa, theta, sigma, rho, v0, r, q, trap)
41     psi = np.exp(-r*tau)*f2/(alpha**2+alpha-v**2+i*v*(2*alpha+1))
42
43     x = np.exp(i*(b-s0)*v)*psi*w
44     e = np.fft.fft(x)
45
46     prices = eta*np.exp(-alpha*k)/np.pi*np.real(e)
47     strikes = np.exp(k)
48
49     return strikes, prices

```

## C.2 Neural Network Architecture

```
1 import torch
2 import torch.nn as nn
3
4 class HestonNN(nn.Module):
5     def __init__(self, input_size=9, hidden_size=128, output_size
6         =1):
7         super().__init__()
8         self.model = nn.Sequential(
9             nn.Linear(input_size, hidden_size),
10            nn.ReLU(),
11            nn.Linear(hidden_size, hidden_size),
12            nn.ReLU(),
13            nn.Linear(hidden_size, hidden_size),
14            nn.ReLU(),
15            nn.Linear(hidden_size, hidden_size),
16            nn.ReLU(),
17            nn.Linear(hidden_size, output_size)
18        )
19
20     def forward(self, x):
21         return self.model(x)
```

## C.3 Training Data Generation

```
1 import numpy as np
2 import heston_fft as hf
3
4 def random_parameters():
5     tau = np.exp(np.random.uniform(np.log(0.01), np.log(2)))
6     while True:
7         kappa = np.random.uniform(0.5, 5)
8         theta = np.random.uniform(0.01, 0.16)
9         sigma = np.random.uniform(0.1, 0.8)
10        if 2*kappa*theta > sigma**2:
11            break
12    rho = np.random.uniform(-0.9, 0.0)
13    v0 = np.random.uniform(0.01, 0.16)
14    r = np.random.uniform(0.01, 0.05)
15    q = np.random.uniform(0, 0.05)
16    return tau, kappa, theta, sigma, rho, v0, r, q
17
18 def generate_data(num_samples, filename):
19     S = 1
20     with open(filename, 'w') as f:
21         for i in range(num_samples):
22             tau, kappa, theta, sigma, rho, v0, r, q =
23                 random_parameters()
24             strikes, prices = hf.heston_call_FFT(4096, 0.25, 1.5, S
25                 , tau, kappa, theta, sigma, rho, v0, r, q, trap=1)
26             lm = np.log(strikes/S)
27             mask = ((lm > -0.5) & (lm < 0.5) & (prices > 0.0001) &
28                 (prices < 0.5))
29             for l, p in zip(lm[mask], prices[mask]):
30                 f.write(f"{l},{tau},{kappa},{theta},{sigma},{rho},{
31                     v0},{r},{q},{p}\n")
```